

Advanced Driver Assistant System using Neuro- Evolutionary trajectory and brain analysis with LIDAR, EEG and AR

by 211055 Khaled Abdelnasser Mahmoud Attaallah Sliem

Submission date: 28-Jun-2024 06:21PM (UTC+0200)

Submission ID: 2409897940

File name:

243350_211055_Khaled_Abdelnasser_Mahmoud_Attaallah_Sliem_Advanced_Driver_Assistant_System_using_Neuro-Evolutionary_431844836.docx (13.25M)

Word count: 11882

Character count: 70094



the
UNIVERSITY
of
GREENWICH

Faculty of Computer Science

Advanced Driver Assistant System using Neuro-Evolutionary trajectory and brain analysis with LIDAR, EEG and AR

Supervised By Dr. Samar Adel

saahmed@msa.edu.eg

Khaled Abdelnasser Mahmoud

211055

Khaled.abdelnasser1@msa.edu.eg

Abstract

This project introduces an Advanced Driving Assistant System which could revolutionize road safety by letting the system attached to cars understand the Risk related to Driving Decision the driver could take. Those Minimizing his bad Decisions which could lead to catastrophic end. While taking approaches of letting reinforcement models incorporate prior knowledge (Traffic Rules) and giving it an interpretability for validation. Also, this project introduces a new integration of analyzing driver's brain signals to predict his actions before its commitment. Those opening a new way of interacting with computer systems throw thinking of that action for validating its safety before the driver even takes it. Those let the system brain connected with driver to predict his actions before its being made.

Furthermore, this work proposes an innovative way to improve present solutions by overcoming one of the most important drawbacks in the manual process of modifying the basic architecture of deep learning networks using the upstart field of neuroevolutionary techniques. In this paper, we introduce an advanced neuroevolutionary algorithm that combines NEAT and DE, searching for the optimal architecture to progress this research area further with new, promising solutions.

Table of Contents

Abstract.....	I
Chapter 1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	2
1.2.1 LIDAR Processing.....	2
1.2.2 Environment Perception.	3
1.2.3 predicting and evaluating best trajectories.	4
1.2.4 Managing uncertainty EEG Integration.....	4
1.2.5 Visualizing Output.	5
1.3 Objective	6
1.4 Motivation.....	6
1.5 Layout.....	7
Chapter 2 Background and Literature Review	8
2.1 Background	8
2.1.1 Reinforcement Learning.....	9
2.1.2 Evolutionary Search	10
2.1.3 Neuro-Evolutionary.....	12
2.1.4 EEG Signal Classification.....	14
2.2 Previous Work	15
2.2.1 Tree-structured Policy Planning with Learned Behavior Models	15
2.2.2 Task-Aware Risk Estimation of Perception Failures for Autonomous Vehicles	18
2.2.3 BAT: Behavior-Aware Human-Like Trajectory Prediction for Autonomous Driving	21
2.2.4 Neuroevolutionary Multi-objective Approaches to Trajectory Prediction in Autonomous Vehicles	24
2.2.5 Cross Talk Neural Architecture Search for EEG Signal Classification	27
Chapter 3: Material and Methods	29
3.1 Materials	29
3.1.1 Data.....	29
3.1.2 Tools.....	30
3.1.3 Environment.....	31

3.2 Methods	31
3.2.1 System architecture Overview	31
3.2.2 Algorithm Description	32
Chapter 4: System Implementation	41
4.1 System Development	41
4.2 System Structure.....	42
4.2.1 System Overview.....	43
4.2.2 Source Code	47
4.3.1 Component A: Car Components	57
4.3.2 Component B: System Components	58
4.3.3 Component C: User Components	60
4.3.4 Final Output	61
Chapter 5 Results and Evaluation	62
5.1 Testing Methodology	62
5.2 Results	62
5.2.1 Best Result Case	63
5.2.2 Limitations.....	63
5.3 Evaluation	64
5.3.1 Accuracy	64
5.3.2 Time Performance.....	64
Chapter 6 Conclusion and Future Work.....	69
6.1 Conclusion.....	69
6.2 Problem Issue.....	70
6.2.1 Technical Issues.....	70
6.2.2 Scientific Issue	71
6.3 Future Work	72
6.3.1 Enhanced Sensor Fusion	72
6.3.2 Model Optimization	72
6.3.3 Improved Interpretability	72
6.3.4 Robustness and Generalization	72
6.3.5 Integration with Autonomous Systems	72
6.3.6 User Interface Enhancements.....	73
6.3.7 Introducing Skip Connections with NEAT	73

6.3.8 potential linking between CNN and GNN	73
References	74

Table of Figures

Figure 1 incoming headlight challenge which RGB is vulnerable to it	2
Figure 2 camera change in lighting causes blindness that lidar is not affected with it	3
Figure 3 example show box-bounding from point cloud data	3
Figure 4 uncertainty-aware safe planning	4
Figure 5 section from the book (how THE BRAIN works 2020 page 167).....	5
Figure 6 examples of AR output	6
Figure 7 Comparison Between Different kind of learning	10
Figure 8 NETWORK ARCHITECTURE AND ITS ENCODING USED BY EVOLUTIONARY TECHNIQUES	11
Figure 9 Classification of nature inspired algorithms	12
Figure 10 Number of Papers Published on Evolutionary Design of Neural Network Architectures (1989–2020)	13
Figure 11 Phenotype and Genotype used by NEAT	14
Figure 12 some of the feature extraction, feature selection and classification techniques used in EEG BCIs.....	15
Figure 13 Structure of TPP	16
Figure 14 Perception Monitor	19
Figure 15 Task-Aware Risk Estimation RESULTS.	19
Figure 16 MOMENTUM-SHAPED DISTANCE CONFUSION MATRIX.....	20
Figure 17 : Architecture of behavior-aware trajectory prediction model	22
Figure 18 Evaluation results for BAT	23
Figure 19 Evaluation results for the proposed model and the baselines in maneuver-base test set for NGSIM dataset.	23
Figure 20 Objective Functions used	25
Figure 21 Diagram of Network Topology developed using Neuro-Evolution	25
Figure 22 : Metric mean and standard deviation (std) for all runs of a given experiment.....	26
Figure 23 CTNASS-EEG Meta-Net.....	27
Figure 24 CTNASS-EEG Search Space	27
Figure 25 CTNAS-EEG Subject dependent performance.....	29
Figure 26 non-invasive EEG.....	29
Figure 27 ATG4D	30
Figure 28 LIDAR Vs Camera Data From Beamng.....	31
Figure 29 System architecture Overview	32
Figure 30 NEAT genome scheme	34
Figure 31 structural mutations of NEAT	35
Figure 32 Crossover of NEAT.....	36
Figure 33 Algorithm of Speciation	36

Figure 34 DE optimizing the 2D ackley function	37
Figure 35 Types of Genetic Operators	38
Figure 36 Method Proposed to overcome variable length chromosomes	38
Figure 37 Loading LIDAR data From BEAMNG.TECH	43
Figure 38 Initialization pseudocode	44
Figure 39 Donor Vector generating using mutation operation	45
Figure 40 Trial Vector generating using crossover operation	45
Figure 41 Variable Encoding length gene	45
Figure 42 gene encoding the cnn architecture	46
Figure 43 examining crossover	46
Figure 44 examining mutation	46
Figure 45 Full Neural architecture search implementation	55
Figure 46 Pseudo Code for evolving a Trajectory prediction on Point Cloud	55
Figure 47 output when fusing with BEAMNG.TECH simulated LIDAR	56
Figure 48 Results conducted for evolving population over 5 generations	62
Figure 49 best scoring candidate	63
Figure 50 accuracy box plots after evolving candidates on each dataset	64

Chapter 1 Introduction

1.1 Introduction

Prediction of the future was one of humanities quests throw ages, humans had reached that any human related activities seem to have a stochastic behavior, but order can be found in chaos. This has given us a reason to study these seemly related events. Driving can be defined as one of the most human related activities, also one of the causes of dying yearly. The fact that humans are part of that system makes it more vulnerable to errors which seem to be fatal. Accidents have lots of types and scenarios, but nearly all share the influence of human error. Those finding a way to prevent could save a lot of lives.

If we think that driving is a complex game of chess where both players share their pieces (actions) and concealing their intentions (strategies) finally all players share the same road (board), But it's not impossible to predict their best moves regarding their infinite possibilities. So, predicting the future could be estimate by analyzing the environmental facts (road condition or other decision related evidence) and preventing the bad decision as it will be a matter of taking the wrong moves at the wrong time. Those achieving a more comfortable and safer path, avoiding, or mitigating the risk of collision.

Yet, what if we could go a step further and predict a driver's actions before they are even committed? Instead of letting them make a potentially harmful decision, advancements in neurotechnology now enable us to predict this action based on analyzing brain signals. This groundbreaking approach will let us proactive intervention, alerting drivers to potential consequences before detrimental action is taken.

1.2 Problem Statement

Driver Assistant Systems plays a big role in ensuring the safety of roads, but lots of challenges nowadays faces the development of that systems, we have defined it as a set of small problems in which if we could solve them efficiently, we can assure that our system will work as needed. The challenges are defined as environmental perception, trajectory prediction and planning, managing uncertainty with EEG integration, and finally visualizing the output. Considering the time sensitivity of evaluating these tasks dynamically to assure driver's safety.

1.2.1 LIDAR Processing.

By comparing LIDAR 3D Object Detection with Computer Vision techniques as discussed by (Bharilya & Kumar, 2023) in his comprehensive survey about challenges facing autonomous vehicles, we concluded that LIDAR's point cloud is much more accurate than predicting depth from 2D images using triangulate algorithms for depth estimation, Also Computer Vision techniques raises the same problem we trying to solve like (Incoming Headlight Blindness, Detection on dark areas, tunnel entering or exiting, etc.). Those taking the point cloud is much better than estimating depth from RGB cameras.



FIGURE 1 INCOMING HEADLIGHT CHALLENGE WHICH RGB IS VULNERABLE TO IT



FIGURE 2 CAMERA CHANGE IN LIGHTING CAUSES BLINDNESS THAT LIDAR IS NOT AFFECTED WITH IT

1.2.2 Environment Perception.

To face the problem of detecting road and other vehicles, as it's a challenge for computer to detects (cars, pedestrians, road conditions, and junctions), Therefore an out of box approach will be needed to face altogether these problems efficiently, also we need a method to take car inputs (steering wheel, acceleration, brakes, etc.) and the approach for this problems will be covered later, also we need to take in consider that collecting dataset of different type of object will be an exhaustive task.

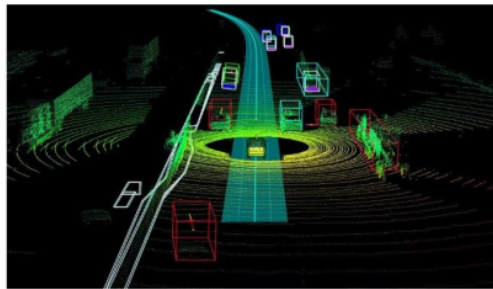


FIGURE 3 EXAMPLE SHOW BOX-BOUNDING FROM POINT CLOUD DATA

1.2.3 predicting and evaluating best trajectories.

In this context, the complex task of predicting and evaluating the best trajectories for multiple objects within the environment, tending to raise some computational issues, As determining the precise future location of each object in the next few seconds may appear impossible due to uncertainties of those predictions as mentioned before due to the stochastic nature of that problem. But the environment will play a major role in filtering this uncertainty, as objects seem to move to their best path. This will give us insight into where they seem to be in the next few seconds. Those we need a way to interrupt our predicted trajectories of them whenever a new uncalculated trajectory seems to emerge, due to the timing we factor we need to achieve our methodology will aim to reduce that uncertainty into a manageable set of trajectories upon which informed decisions can be based on.

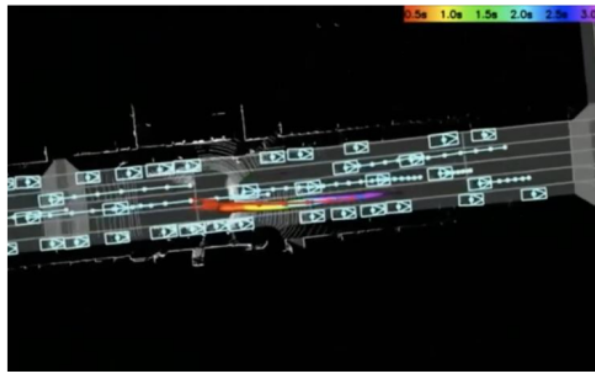


FIGURE 4 UNCERTAINTY-AWARE SAFE PLANNING

1.2.4 Managing uncertainty EEG Integration.

As discussed in the previous part, driving seems to have its stochastic behavior. In this context we aim to reduce the uncertainty of the driver's ego motion by analyzing his brain signals. If we could achieve prediction of driver intended action or even the action he's taking, as neuro-analysis of brain signals show that motor actions (braking, accelerating, etc.) seems to fire a sequence of neurons 1sec

before the real action is taking place. So, here we could uncover the uncertainty of our driver's intention. Those saving time validate his pre-action and warn him of its bad consequences. Of course, this module will be optional as driving with EEGs will bother the users, but we hope that integrating this EEG with the next AR module will be the key to ignoring this issue. And knowing that EEG signals is biased to its currently user, a gamified training will be made to let the signal classifier collect user's related brain signals to different scenarios.

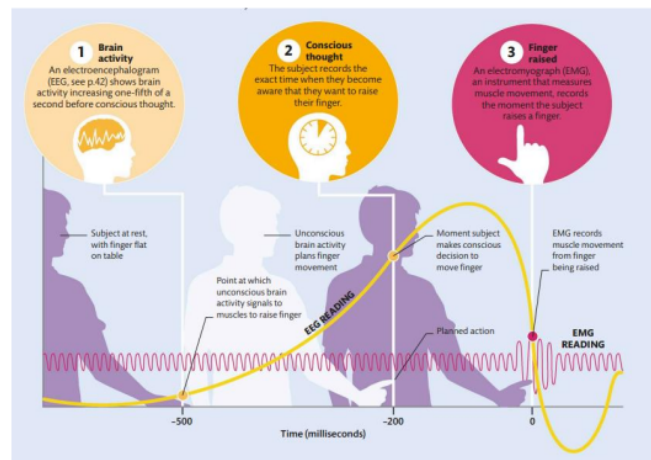


FIGURE 5 SECTION FROM THE BOOK (HOW THE BRAIN WORKS 2020 PAGE 167)

1.2.5 Visualizing Output.

Our first approach was to make an AR-HUD which will present our drawing pixels to the Windshield, but this raises the issue of showing blind spots of the driver; Those we thought of using AR Glasses as our final Solution to Give the driver the full functionality and open the potential for more content to be shown (LIDAR real-time view, Parking assist, GPS road navigation, path planning, obstacle detection, Road Analysis, etc.).

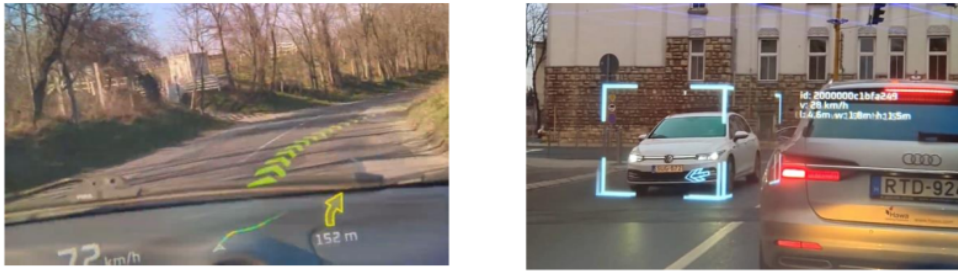


FIGURE 6 EXAMPLES OF AR OUTPUT

1.3 Objective

This project should present a fully integrable system for cars, in which it could handle lots of environmental factors and help drivers in taking the best decisions, also evaluating their decisions for reducing the risk of human errors. Alternatively, this project should include an optional module for brain-computer interfacing which will revolutionize the driving process. Those opening the gate for brain-connecting vehicles in which the driver interfaces with the car through his thoughts, which we thought will decrease the rate of accidents by allowing the driver to focus more on his primary task. Also, wrapping the output using non-disturbing Augmented Reality Techniques.

1.4 Motivation

Around 94% of road accident is happening due to human error as mentioned by Click or tap here to enter text.(Bharilya & Kumar, 2023) in his survey and this number is increasing as number of drivers increases yearly, our project aims to reduce this number of human's errors by helping its users taking correct decisions or alerting them to escape accidents safely. Also, our project aims to implement a generic road model that not only obeys the traffic rules, but it could also calculate when to escape these rules to safely minimize the risk of accidents. However, this project will introduce a new model

that will learn to navigate of Egyptian harsh environment, where nearly no-one at a point will obey the rules and this is our main objective, alongside saving human lives.

1.5 Layout

This thesis is further divided into six main chapters. An overview of each chapter is presented below:

- Chapter 1 Introduction.
- Chapter 2 Background and Literature Review.
- Chapter 3 Material and Methods.
- Chapter 4 System implementation.
- Chapter 5 Results and Evaluation.
- Chapter 6 Conclusion and Future Work.

Chapter 2 Background and Literature Review

2.1 Background

The problem rises since over 100 years ago, it dates back from 1925 when the first “**radio-controlled car**” had been made, fast forward to 1940 an innovating highway designed for radio-controlled cars, and this gives hope that in 1960 human will be completely removed from the driving process. Through 1950s the first approach for autonomous vehicles was made by embedding wires in road to make their car percept their environment, but their approach required roads to be made specially to engage with that system, Unfortunately the theorized that this system will be in commercial use by 1975, But this never happen.

As the space project began, the development of a remote system that processes cart images to decide which route to take, a remote processing unit was made, and this was the initiation of autonomous vehicles. Later in 1980 Mercedes Benz robotic van was the first vehicle to introduce on-board modular processing unit, later in that decade the first use of **neural networks** in steering and navigation process. Through the years Machine learning was the take to solve the problems of automated driving. In 2010 top car manufacturers had been challenging each other to pioneer in TAP (**Temporary Auto Pilot**) and Audi was the one who beats the competition to set the world record at the time of driverless top speed driving.

By attaching neural network weights with car properties like accelerating and steering this will raise the need of simulating the environment to get policy-based behavior (penalty and reward) to make the neural network learn to drive well, and this the challenge since the 90s required Computer Vision to solve the environment perception, But this approach through years had shown its lake of accuracy because it try to get depth perception from 2D images, which is affected by many factors (weather, lighting), Also there's the problem that computer vision will recursively detect object within the context

of other object (man riding a bicycle), LIDARS nowadays aims to fix the vulnerabilities of 2D images, by transmitting pulses of Laser that later will be used to calculate the distance of each point those generating our point cloud with 3D depth data calculated without the need for time consuming depth estimation algorithms.

In 2018 Nissan introduced the term Brain-Connected Ready Vehicles, which we aim to connect the driver's brain to EEG unit to interpret his brain signals to determine what action the drive could take, as they are exploited that motor cortex reveals brain activity 0.5 seconds before the real action. Those helping the driver handle the car better, this was the first time AI is working alongside human drivers not replacing them. Also, this is shown in Adaptive Cruise Control modules as cars seem to let us move away from accidents, they theorize that their technology will be available for consumers in the next 10 years.

2.1.1 Reinforcement Learning

The term Reinforcement Learning (RL) describes a class of machine learning that enables an agent to learn through interaction with an environment. The agent starts with minimum to no prior knowledge of how to behave in the environment and must learn to take actions that maximize a reward signal based on the outcomes of previously selected actions. The agent iteratively refines its predefined policy (π) to reach some optimal control policy (π^*) by receiving feedback from the environment in the form of a scalar reward signal during its training phase.

The RL agent's main objective is to maximize the reward signal it receives during its lifetime by finding a balance between exploiting known good actions and exploring the consequences of new actions. This balance is known as the exploration-exploitation trade-off, and it is a key aspect of RL.

RL problems are generally formulated as Markov Decision Processes (MDPs) as mentioned by (W. Chen et al., 2023). MDPs consist of a set of states(S), a transition function (T), a set of actions(A), and a reward function(R) represented as a tuple $\langle S, T, A, R \rangle$. The state of the environment at any given time

is the agent's perception of the environment, and the transition function determines the probability of moving from one state to another given action. The set of available actions represents the possible decisions the agent can take at any given state, and the reward function specifies the reward the agent receives for taking a specific action in a particular state.

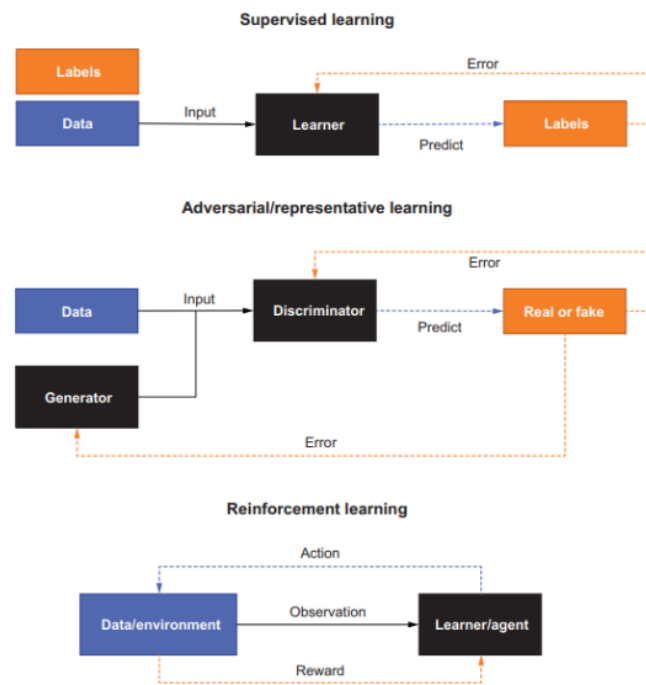


FIGURE 7 COMPARISON BETWEEN DIFFERENT KIND OF LEARNING

2.1.2 Evolutionary Search

Optimization is the process of finding the best combination for a set of decision variables to solve a certain problem. Optimization arises in many fields, different disciplines, and countless applications (Talbi, 2009). These decision variables can be mapped into later as a weight of a neural activation function or a string in a genetic sequence used to describe a model that can evolve to fit his required problem.

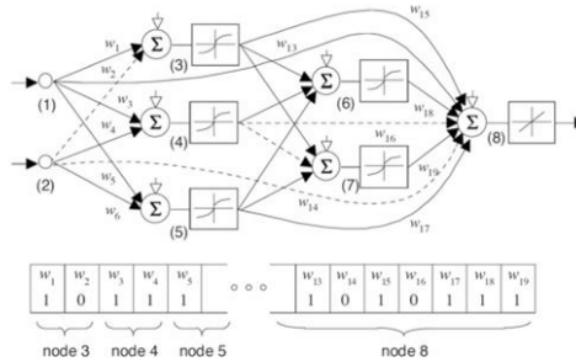


FIGURE 8 NETWORK ARCHITECTURE AND ITS ENCODING USED BY EVOLUTIONARY TECHNIQUES

These algorithms fall into two categories: approximation methods and accurate methods. It is ensured that exact approaches will yield an optimal solution in a reasonable amount of time, unless the problem is categorized as NP-Hard, meaning that no polynomial time exists. This results in extremely long computation times. The term "metaheuristic" refers to a recently established class of approximate algorithms. Over the past three decades, metaheuristics have been utilized extensively due to its simplicity, ease of application, ability to avoid local optima, and capacity to handle derivative-free situations. The metaheuristic's two key features are exploitation and exploration. The first one shows that the algorithm can locate new search areas, and the second one concentrates on locating the best answer in a promising area. The metaheuristic that can offer the right amount of exploration and exploitation is the one that will work. Different categories of metaheuristics by (Blum, 2003) can be found in the literature. Nature-inspired versus non-nature-inspired, population-based versus single-point search, dynamic versus static objective functions, one neighborhood versus multiple neighborhoods, and memory-intensive versus memory-light techniques are some examples of these differences. (Mohamed et al., 2020) will subsequently divide Natural Inspired Algorithms into four categories. shown in **FIGURE 9**.

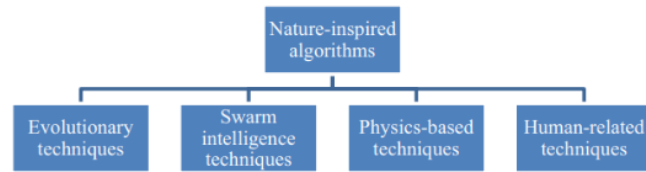


FIGURE 9 CLASSIFICATION OF NATURE INSPIRED ALGORITHMS

Combining Evolutionary Algorithms and Reinforcement Learning is possible because the two approaches are not mutually exclusive, just like the natural principles they are based on can coexist. Reinforcement Learning enables agents to make decisions based on a reward function. However, during the learning process, the choice of parameter values for the learning algorithm can greatly affect the overall performance. Using an Evolutionary Algorithm to determine these parameter values, for instance in the Deep Deterministic Policy Gradient (DDPG) method combined with Hindsight Experience Replay (HER), can accelerate the learning process, leading to improved performance faster than the original algorithms. Another approach is to incorporate aspects of Reinforcement Learning, such as the Agent-Environment relationship, and apply Evolutionary Algorithm techniques like crossover and mutation to multiple agents. This approach was utilized by (Humeniuk et al., 2023) in their research, where they merged both techniques to create RIGAA (Reinforcement Learning Informed Genetic Algorithm for Autonomous Systems Testing). This integration opens up new promising possibilities in the field of Evolutionary Search.

2.1.3 Neuro-Evolutionary

Neuro-Evolutionary is a recently established discipline that applies evolutionary computation methods to network design or training in artificial neural networks (Stanley et al., 2019), as he is the one introduces the famous NEAT algorithm (NeuroEvolution of Augmenting Topologies) (Stanley & Miikkulainen, 2002). In Their research Ünal & Başçiftçi (2022) examined the historical development of

neural architecture evolution over the past three decades (1989-2020). They divided this timeframe into three distinct periods based on significant achievements and trends. The first period includes the initial efforts to evolve simple ANN architectures competitively to create efficient chromosome representation strategies. The second period begins with Stanley's introduction of NEAT in 2002, which is considered a major breakthrough and milestone in the field. During this period, there were numerous attempts to enhance or surpass NEAT in various ways. The third period corresponds to the Deep Learning Era, where researchers investigated methods to automate the design and configuration of deep neural networks. Later Major Updates on the NEAT had been done to update it to deal with more complex Deep Learning architectures like (Bohrer et al., 2020) CoDeepNEAT with had a clever way of developing micro and macro architecture to evolve complex networks. Lately a new NEAT alternative shown by (L. Wang et al., 2024) which he had tensorized the NEAT showing that after 22 years of its original implementation its still an active area of research.

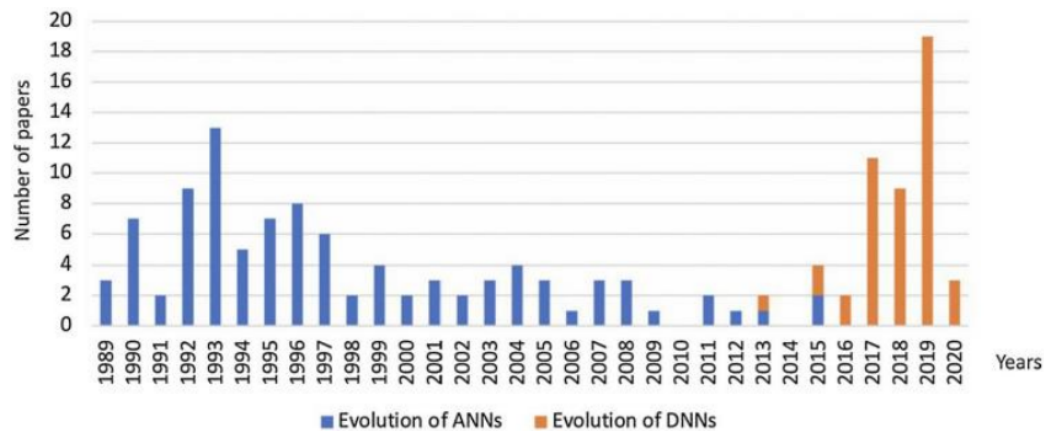


FIGURE 10 NUMBER OF PAPERS PUBLISHED ON EVOLUTIONARY DESIGN OF NEURAL NETWORK ARCHITECTURES (1989–2020)

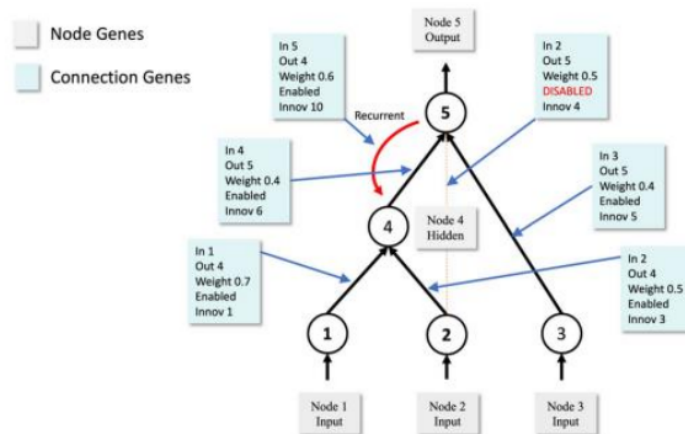


FIGURE 11 PHENOTYPE AND GENOTYPE USED BY NEAT

For more information about the universal approximation theorem; which also give the need to develop a better neuro evolutionary technique to find the best architecture for any require problem; the following research gives a better insight into.

- (Cybenkot, 1989) Approximations by Superpositions of Sigmoidal Functions.
- (Hornik, 1991) Approximation Capabilities of Multilayer Feedforward Networks.
- (Leshno et al., 1992) Multilayer feedforward networks with a nonpolynomial activation function can approximate any function.
- (Hanin & Sellke, 2017) Approximating Continuous Functions by ReLU Nets of Minimal Width.

2.1.4 EEG Signal Classification

Signal Classification is a classical machine learning problem in which lots of classifiers happens to solve it with accuracies higher 90% as shown by (Padfield et al., 2019), interestingly a new method was proposed by (Baig et al., 2017) as he used Differential Evolution (DE) an Evolutionary Search technique

as a method for feature extraction which is a preprocessing task while working with RAW EEG data as shown in FIGURE 12.

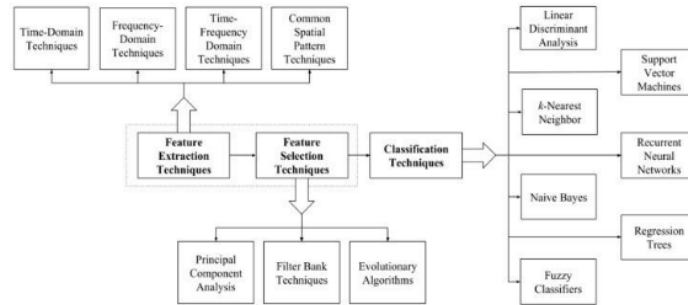


FIGURE 12 SOME OF THE FEATURE EXTRACTION, FEATURE SELECTION AND CLASSIFICATION TECHNIQUES USED IN EEG BCIS

This let us question if a better evolutionary techniques will help us leveraging the accuracy of that classification in promising ones, and our choice was Neuro-Evolutionary Augmented Topologies an Algorithm introduced by (Stanley & Miikkulainen, 2002) as it shows promising results given less computation power and fast time, as it works by evolving the neural network topology until no better results could achieved, this is computationally better than working with a fully connected neural network.

2.2 Previous Work

2.2.1 Tree-structured Policy Planning with Learned Behavior Models

2.2.1.1 Strategy & Structure

(Y. Chen et al., 2023) describe the Tree-Structured Policy Planning (TPP) model, which addresses the challenges of motion planning for autonomous vehicles by constructing two tree structures: a multi-stage prediction model and a dynamic programming approach to solve for the optimal policy. The TPP model comprises three main components: an ego motion sampler that generates the ego trajectory

tree, a deep-learned multi-stage prediction model for scenario tree generation, and a dynamic programming module to determine the optimal policy based on the constructed Markov Decision Process (MDP). The ego motion sampler produces dynamically feasible trajectory segments representing potential trajectories for the ego vehicle. The prediction module, utilizing a deep-learned model, provides multi-modal predictions for the environment. Finally, the dynamic programming module translates the trajectory and scenario trees into an MDP and employs dynamic programming to find the optimal policy.

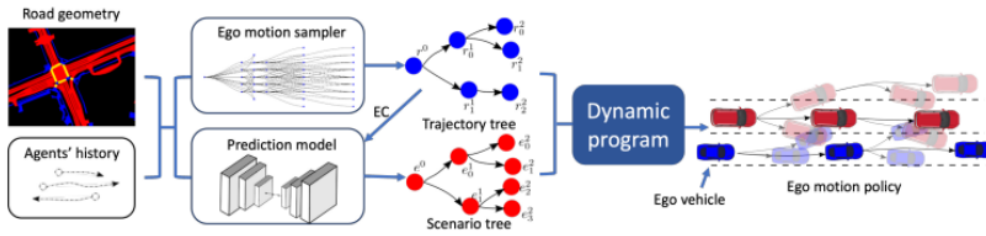


FIGURE 13 STRUCTURE OF TPP

2.2.1.2 Data

Data is collected in real-time after being evaluated for object detection and tracking using or LIDAR preprocessing, as well as no dataset is used for training as it's an RL approach.

2.2.1.3 Method Evaluation

The TPP model is designed with the following key principles in mind:

1. It is compatible with cutting-edge deep-learning models used for trajectory forecasting.
2. It plans with closed-loop ego-conditioning, meaning it takes into account how the ego vehicle's actions impact other agents on the road.

Method	Reference	Deep learning	Ego-conditioning	Interpretable	Multistage	Scalability
Multi-Mode planning	[1]-[4]	No	No	Yes	No	Poor
Branch MPC	[5]	No	Yes	Yes	Yes	Poor

POMDP	[6]-[8]	No	Yes	Yes	Yes	Poor
Game Theoretic	[9],[10]	No	Yes	Yes	No	Very Poor
Lookout	[11]	Yes	No	Yes	No	Good
CFO	[12]	Yes	Yes	No	No	Medium
TPP	[13]	Yes	Yes	Yes	yes	Good

Number	Reference
1	J. Hardy and M. Campbell, "Contingency planning over probabilistic hybrid obstacle predictions for autonomous road vehicles," in 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2010, pp. 2237–2242.
2	—, "Contingency planning over probabilistic obstacle predictions for autonomous road vehicles," IEEE Transactions on Robotics, vol. 29, no. 4, pp. 913–929, 2013.
3	. Zhan, C. Liu, C.-Y. Chan, and M. Tomizuka, "A non-conservatively defensive strategy for urban autonomous driving," in 2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC). IEEE, 2016, pp. 459–464.
4	. Galceran, A. G. Cunningham, R. M. Eustice, and E. Olson, "Multipolicy decision-making for autonomous driving via changepoint-based behavior prediction: Theory and experiment," Autonomous Robots, vol. 41, no. 6, pp. 1367–1382, 2017.
5	Y. Chen, U. Rosolia, W. Ubellacker, N. Csomay-Shanklin, and A. D. Ames, "Interactive multi-modal motion planning with branch model predictive control," IEEE Robotics and Automation Letters, vol. 7, no. 2, pp. 5365–5372, 2022.
6	H. Bai, S. Cai, N. Ye, D. Hsu, and W. S. Lee, "Intention-aware online pomdp planning for autonomous driving in a crowd," in 2015 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2015, pp. 454–460.
7	W. Liu, S.-W. Kim, S. Pendleton, and M. H. Ang, "Situation-aware decision making for autonomous driving on urban road using online pomdp," in 2015 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2015, pp. 1126–1133.
8	T. Bandyopadhyay, K. S. Won, E. Frazzoli, D. Hsu, W. S. Lee, and D. Rus, "Intention-aware motion planning," in Algorithmic foundations of robotics X. Springer, 2013, pp. 475–491.
9	J. F. Fisac, E. Bronstein, E. Stefansson, D. Sadigh, S. S. Sastry, and A. D. Dragan, "Hierarchical game-theoretic planning for autonomous vehicles," in 2019 International conference on robotics and automation (ICRA). IEEE, 2019, pp. 9590–9596.
10	D. Sadigh, S. Sastry, S. A. Seshia, and A. D. Dragan, "Planning for autonomous cars that leverage effects on human actions." in Robotics: Science and systems (RSS), vol. 2. Ann Arbor, MI, USA, 2016, pp. 1–9.
11	A. Cui, S. Casas, A. Sadat, R. Liao, and R. Urtasun, "Lookout: Diverse multi-future prediction and planning for self-driving," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 16 107–16 116.

12	N. Rhinehart, J. He, C. Packer, M. A. Wright, R. McAllister, J. E. Gonzalez, and S. Levine, "Contingencies from observations: Tractable contingency planning with learned behavior models," arXiv preprint arXiv:2104.10558, 2021.
13	Chen, Y., Karkus, P., Ivanovic, B., Weng, X., & Pavone, M. (2023). <i>Tree-structured Policy Planning with Learned Behavior Models</i> . http://arxiv.org/abs/2301.11902

2.2.2 Task-Aware Risk Estimation of Perception Failures for Autonomous Vehicles

2.2.2.1 Strategy & Structure

(Antonante et al., 2023) developed a task-aware risk estimator for autonomous vehicles to assess the risk that perception failures pose against the AV's motion plan. The approach leverages trajectory prediction networks and utilises the statistical tool of copula in estimating relative scenario risk, providing probably approximately correct estimates of risk. It also proposes a task-aware perception monitor, comprising three components: perception failure detection and identification, plausible scene generator, and task-aware risk estimator. The authors present a detection algorithm that, based on the RSR PAC bounds, raises an alarm in the presence of high-risk, task-relevant failures. Experimental results demonstrate performance superior to established baselines on the NuPlan dataset.

The strategy applied in the following study is to formalize the task-aware risk estimation problem, develop an algorithm that can estimate RSR at runtime, and compare this proposed method with prior approaches in realistic perception failure scenarios. The structured approach will cover: an overview of the task-aware perception monitor; a detailed discussion of the perception failure detection

and identification module; and the formalization of relative scenario risk.

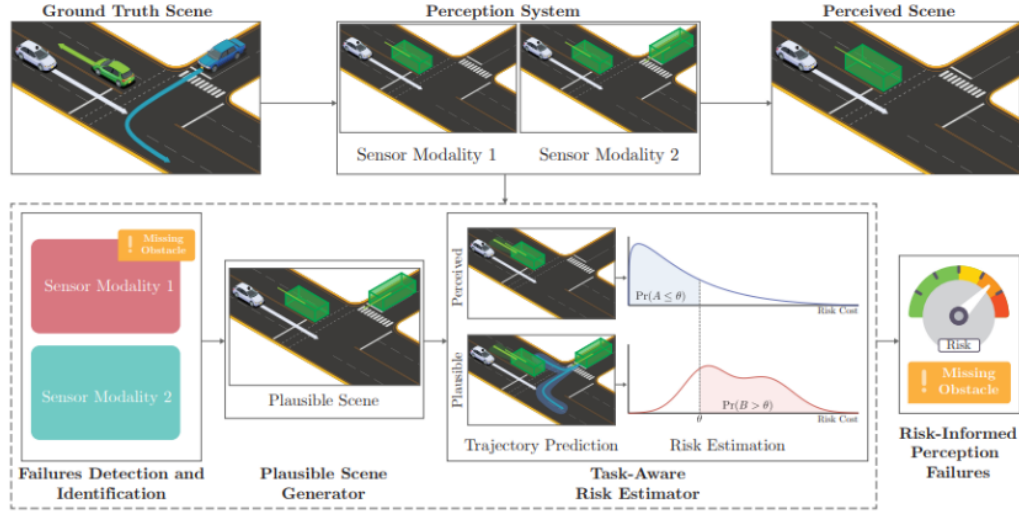


FIGURE 14 PERCEPTION MONITOR

2.2.2.2 Data

NuPlan dataset for testing the proposed approach. The dataset includes 100 realistic perception failure scenarios, each categorized into four classes of failures: Misdetection, Missed Obstacle, Ghost Obstacle, and Mis-localization, with various subclasses within each class. The dataset is used to implement a fault injection mechanism and evaluate the risk estimation.

the document mentions the use of the Intelligent-Driver Model (IDM) planner and Trajectron++ for closed-loop simulations and trajectory prediction, respectively.

2.2.2.3 Method Evaluation

Algorithm	Parameters	F1 Score	Accuracy	Precision	Recall	Alarm-to-Collision [s]		Runtime [s]	
						Average	Median	Average	Median
Momentum-Shaped Distance (Proposed)	$p = 0.90, \gamma = 0.9, \alpha = 0.1$	0.70	0.80	0.55	0.96	5.28	3.60	0.29	0.2
	$p = 0.95, \gamma = 0.9, \alpha = 0.1$	0.79	0.88	0.68	0.96	5.18	3.60		
	$p = 0.99, \gamma = 0.9, \alpha = 0.1$	0.86	0.93	0.81	0.92	4.72	3.03		
Time-To-Collision (Proposed)	$p = 0.90, \gamma = 0.9, \alpha = 0.1$	0.70	0.80	0.55	0.96	4.61	2.45	0.26	0.17
	$p = 0.95, \gamma = 0.9, \alpha = 0.1$	0.76	0.86	0.65	0.92	4.16	2.05		
	$p = 0.99, \gamma = 0.9, \alpha = 0.1$	0.79	0.90	0.79	0.79	4.13	2.25		
Collision Probability	$\gamma = 0.90$	0.40	0.32	0.26	0.96	6.53	4.95	0.25	0.17
	$\gamma = 0.95$	0.41	0.33	0.26	0.96	6.31	4.95		
HJ-Reachability	$\gamma = 0.99$	0.43	0.41	0.28	0.92	4.33	3.48	0.01	0.01
	$\gamma = 0$	0.39	0.28	0.24	0.96	7.10	5.75		

FIGURE 15 TASK-AWARE RISK ESTIMATION RESULTS.

		Predicted	
		High-Risk	Low-Risk
Actual	High-Risk	22 (True Positive)	2 False Negative
	Low-Risk	5 False Positive	71 True Negative

FIGURE 16 MOMENTUM-SHAPED DISTANCE CONFUSION MATRIX

The trajectory prediction module takes on average, 0.22 seconds to complete its calculations, which constitutes about 75% of the total time required for the entire process. This significant time consumption can be reduced by switching to a faster trajectory prediction module, such as PredictionNet. PredictionNet is substantially quicker, being two orders of magnitude faster than the existing module, Trajectron++. This means PredictionNet can perform the same calculations much more rapidly.

Furthermore, the proposed approach can be made even more efficient through parallelization. In parallel computing, multiple calculations or processes are carried out simultaneously, which can drastically reduce the overall computation time. For this specific approach, the cost calculations for each agent in the scene can be done simultaneously rather than sequentially. Additionally, the prediction network can process multiple scenes in a single batch, rather than handling them one by one.

By implementing these improvements—using a faster trajectory prediction module like PredictionNet and parallelizing the cost computations and scene predictions—the overall runtime of the process is expected to decrease significantly, leading to a much faster and more efficient system.

Number	Reference
1	Alexey Kamenev, Lirui Wang, Ollin Boer Bohan, Ishwar Kulkarni, Bilal Kartal, Artem Molchanov, Stan Birchfield, David Nister, and Nikolai Smolyanskiy. Predictionnet: Real-time joint probabilistic traffic prediction for planning, control, and simulation. In Proceedings of International Conference on Robotics and Automation (ICRA), pages 8936–8942, 2022.

2.2.3 BAT: Behavior-Aware Human-Like Trajectory Prediction for Autonomous Driving

2.2.3.1 Strategy & Structure

(Liao et al., 2023) The model integrates insights and findings from traffic psychology and human behavior to enhance trajectory prediction. By understanding how drivers perceive and respond to different scenarios, the model aims to mimic human-like comprehension and response in autonomous vehicles. It consists of multiple modules that address different aspects of trajectory prediction. The behavior-aware module focuses on capturing and representing driving behavior, while the interaction-aware module considers the interactions among vehicles. The priority-aware module accounts for the priority of different vehicles, and the position-aware module incorporates the spatial understanding of the environment.

The model uses a dynamic geometric graph approach, which means we don't need to manually label data during training. This approach effectively captures continuous driving behaviors and tackles the challenges of labeling non-continuous actions and choosing the right time frames. It uses centrality metrics and behavior-aware criteria to ensure flexibility and accuracy in representing driving behavior.

To make the model more intuitive, especially in understanding spatial relationships, it employs a unique pooling mechanism that captures vehicle positions in polar coordinates. This method makes it easier to model direction and distance, takes road curves into account, and is particularly helpful in complex situations like roundabouts and intersections.

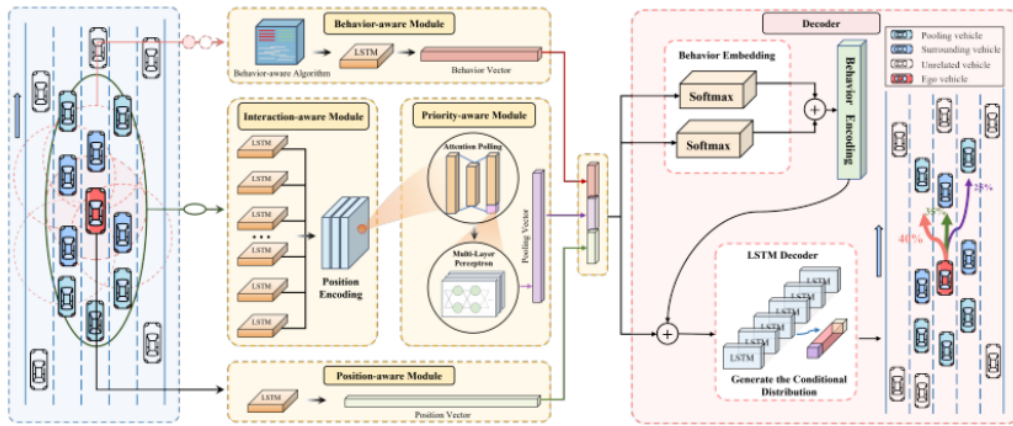


FIGURE 17 : ARCHITECTURE OF BEHAVIOR-AWARE TRAJECTORY PREDICTION MODEL

2.2.3.2 Data

The model's performance is evaluated on various datasets, including NGSIM, HighD, Round, and the newly introduced MoCAD dataset. By testing the model on diverse traffic scenarios, such as highways, roundabouts, and busy urban locales, its robustness and adaptability are assessed.

2.2.3.3 Method Evaluation

Dataset	Model	Prediction Horizon (s)				
		1	2	3	4	5
NGSIM	S-LSTM (Alahi et al. 2016)	0.65	1.31	2.16	3.25	4.55
	S-GAN (Gupta et al. 2018)	0.57	1.32	2.22	3.26	4.40
	CS-LSTM (Deo and Trivedi 2018a)	0.61	1.27	2.09	3.10	4.37
	MATF-GAN (Zhao et al. 2019)	0.66	1.34	2.08	2.97	4.13
	NLS-LSTM (Messaoud et al. 2019)	0.56	1.22	2.02	3.03	4.30
	M-LSTM (Deo and Trivedi 2018b)	0.58	1.26	2.12	3.24	4.66
	IMM-KF (Lefkopoulou et al. 2020)	0.58	1.36	2.28	3.37	4.55
	GAIL-GRU (Kuefler et al. 2017)	0.69	1.51	2.55	3.65	4.71
	MFP (Tang and Salakhutdinov 2019)	0.54	1.16	1.89	2.75	3.78
	DRBP (Gao et al. 2023)	1.18	2.83	4.22	5.82	-
	DN-IRL (Fernando et al. 2019)	0.54	1.02	1.91	2.43	3.76
	WSiP (Wang et al. 2023)	0.56	1.23	2.05	3.08	4.34
	CF-LSTM (Xie et al. 2021)	0.55	1.10	1.78	2.73	3.82
	MHA-LSTM (Messaoud et al. 2021)	0.41	1.01	1.74	2.67	3.83
	HMNet (Xue et al. 2021)	0.50	1.13	1.89	2.85	4.04
	TS-GAN (Wang et al. 2022b)	0.60	1.24	1.95	2.78	3.72
	STDAN (Chen et al. 2022b)	0.39	0.96	1.61	2.56	3.67
	BAT (25%)	0.31	0.85	1.65	2.69	3.87
	BAT	0.23	0.81	1.54	2.52	3.62
HighD	S-LSTM (Alahi et al. 2016)	0.22	0.62	1.27	2.15	3.41
	S-GAN (Gupta et al. 2018)	0.30	0.78	1.46	2.34	3.41
	WSiP (Wang et al. 2023)	0.20	0.60	1.21	2.07	3.14
	CS-LSTM (Deo and Trivedi 2018a)	0.23	0.65	1.29	2.18	3.37
	CS-LSTM (Deo and Trivedi 2018a)	0.22	0.61	1.24	2.10	3.27
	MHA-LSTM (Messaoud et al. 2021)	0.19	0.55	1.10	1.84	2.78
	MHA-LSTM (+f) (Messaoud et al. 2021)	0.06	0.09	0.24	0.59	1.18
	NLS-LSTM (Messaoud et al. 2019)	0.20	0.57	1.14	1.90	2.91
	DRBP (Gao et al. 2023)	0.41	0.79	1.11	1.40	-
	EA-Net (Cui et al. 2021)	0.15	0.26	0.43	0.78	1.32
	CF-LSTM (Xie et al. 2021)	0.18	0.42	1.07	1.72	2.44
	STDAN (Chen et al. 2022b)	0.19	0.27	0.48	0.91	1.66
	iNATran (M) (Chen et al. 2022a)	0.04	0.05	0.21	0.54	1.11
	iNATran (Chen et al. 2022a)	0.04	0.05	0.21	0.54	1.10
	BAT (25%)	0.14	0.34	0.65	0.89	1.27
	BAT	0.08	0.14	0.20	0.44	0.62
Round	S-LSTM (Alahi et al. 2016)	0.94	1.82	3.43	5.21	-
	S-GAN (Gupta et al. 2018)	0.72	1.57	3.01	4.78	-
	CS-LSTM (Deo and Trivedi 2018a)	0.74	1.43	2.44	4.21	-
	CS-LSTM (Deo and Trivedi 2018a)	0.71	1.21	2.09	3.92	-
	MATH (Hasan et al. 2021)	0.38	0.80	1.76	3.08	-
	MHA-LSTM (Messaoud et al. 2021)	0.62	0.98	1.88	3.65	-
	MHA-LSTM (+f) (Messaoud et al. 2021)	0.51	0.91	1.80	3.57	-
	NLS-LSTM (Messaoud et al. 2019)	0.62	0.96	1.91	3.48	-
	WSiP (Wang et al. 2023)	0.52	0.99	1.88	3.07	-
	CF-LSTM (Xie et al. 2021)	0.51	0.87	1.79	3.14	-
	STDAN (Chen et al. 2022b)	0.35	0.77	1.74	2.92	-
	BAT (25%)	0.32	0.72	1.99	3.12	-
	BAT	0.23	0.55	1.43	2.46	-
MoCAD	S-LSTM (Alahi et al. 2016)	1.73	2.46	3.39	4.01	4.93
	S-GAN (Gupta et al. 2018)	1.69	2.25	3.30	3.89	4.69
	CS-LSTM (Deo and Trivedi 2018a)	1.49	2.07	3.02	3.62	4.53
	CS-LSTM (Deo and Trivedi 2018a)	1.45	1.98	2.94	3.56	4.49
	MHA-LSTM (Messaoud et al. 2021)	1.25	1.48	2.57	3.22	4.20
	MHA-LSTM (+f) (Messaoud et al. 2021)	1.05	1.39	2.48	3.11	4.12
	NLS-LSTM (Messaoud et al. 2019)	0.96	1.27	2.08	2.86	3.93
	WSiP (Wang et al. 2023)	0.70	0.87	1.70	2.56	3.47
	CF-LSTM (Xie et al. 2021)	0.72	0.91	1.73	2.59	3.44
	STDAN (Chen et al. 2022b)	0.62	0.85	1.62	2.51	3.32
	BAT (25%)	0.65	0.99	1.89	2.81	3.58
	BAT	0.35	0.74	1.39	2.19	2.88

FIGURE 18 EVALUATION RESULTS FOR BAT

AND THE BASELINES IN THE OVERALL TEST SET OVER A DIFFERENT HORIZON.

NOTE: RMSE (M) IS THE EVALUATION METRIC, WHERE LOWER VALUES INDICATE BETTER PERFORMANCE, WITH SOME NOT SPECIFYING ('-'). VALUES IN BOLD REPRESENT THE BEST PERFORMANCE IN EACH CATEGORY

interaction-aware and priority-aware modules significantly boosted performance, underlining their importance in enhancing prediction accuracy. The behavior-aware module's inclusion significantly enhanced performance by capturing dynamic vehicular interactions, vital for accurate trajectory

Dataset	Model	keep					merge				
		Horizon (s)					Horizon (s)				
		1	2	3	4	5	1	2	3	4	5
NGSIM	S-LSTM (Alahi et al. 2016)	0.35	1.01	1.81	2.82	4.15	0.81	1.31	2.51	4.01	5.78
	S-GAN (Gupta et al. 2018)	0.36	1.01	1.81	2.83	4.15	0.71	1.32	2.53	4.11	5.97
	CS-LSTM (Deo and Trivedi 2018a)	0.34	0.98	1.75	2.77	4.06	0.61	1.34	2.58	4.12	5.94
	MATF-GAN (Zhao et al. 2019)	0.37	1.11	1.74	2.66	3.91	0.53	1.41	2.56	3.97	5.52
	WSiP (Wang et al. 2023)	0.32	0.89	1.58	2.51	3.59	0.40	1.18	2.41	3.72	5.16
	HMNet (Xue et al. 2021)	0.31	0.83	1.56	2.51	3.68	0.34	1.17	2.32	3.63	5.20
	STDAN (Chen et al. 2022b)	0.28	0.85	1.52	2.53	3.49	0.28	1.19	2.21	3.67	4.95
	BAT (25%)	0.28	0.86	1.54	2.52	3.73	0.31	0.95	1.95	3.31	4.98
	BATa	0.23	0.81	1.49	2.44	3.66	0.25	0.89	1.83	3.04	4.45
HighD	S-LSTM (Alahi et al. 2016)	0.77	1.68	3.04	4.67	6.59	0.69	1.97	3.81	6.17	9.09
	S-GAN (Gupta et al. 2018)	0.66	1.68	3.11	4.85	6.87	0.72	1.97	3.91	6.32	9.23
	CS-LSTM (Deo and Trivedi 2018a)	0.54	1.63	3.01	4.71	6.63	0.61	2.01	3.97	6.48	9.48
	MATF-GAN (Zhao et al. 2019)	0.61	1.72	3.02	4.62	6.34	0.56	1.88	3.90	6.07	9.01
	WSiP (Wang et al. 2023)	0.41	1.46	2.82	4.42	6.22	0.52	1.61	3.60	5.78	8.45
	HMNet (Xue et al. 2021)	0.41	1.31	2.87	4.47	6.33	0.49	1.62	3.47	5.87	8.59
	STDAN (Chen et al. 2022b)	0.35	1.33	2.84	4.51	5.97	0.38	1.49	3.46	5.87	7.93
	BAT (25%)	0.43	1.24	2.43	4.01	5.91	0.47	1.41	3.09	5.19	7.87
	BAT	0.33	1.07	2.24	3.73	5.51	0.31	1.36	2.96	5.15	6.78

FIGURE 19 EVALUATION RESULTS FOR THE PROPOSED MODEL AND THE BASELINES IN MANEUVER-BASE TEST SET FOR NGSIM DATASET.

prediction. By factoring in surrounding vehicles' behavior, BAT predicts the likely future trajectories with higher precision and reliability. This module enables the system to anticipate potential lane changes, overtaking maneuvers, and speed adjustments of nearby vehicles, leading to more human-like and realistic trajectory predictions. Consequently, BAT can better navigate complex driving scenarios and improve overall safety and efficiency in autonomous driving environments. This comprehensive approach demonstrates the critical role of incorporating behavior-aware dynamics into trajectory prediction models, ensuring more accurate and reliable predictions essential for advanced autonomous driving systems.

2.2.4 Neuroevolutionary Multi-objective Approaches to Trajectory Prediction in Autonomous Vehicles

2.2.4.1 Strategy & Structure

(Stapleton et al., 2022) employs a neuroevolutionary multi-objective optimization (EMO) approach to improve trajectory prediction for autonomous vehicles. The key strategy involves:

Network Composition where they used a combination of Convolutional Neural Networks (CNNs) and Long-Short Term Memory (LSTM) to leverage both temporal and spatial data features. Alongside **optimization algorithm** which employs non-dominated Sorting Genetic Algorithm II (NSGA-II) to optimize the hyper parameters of the CNN-LSTM network, which allows the simultaneous optimization of multiple conflicting objectives. Finally, **Objective Functions** which works by evaluating five objectives to understand their impact on the neuroevolutionary process. These objectives include distance feedback, lateral velocity longitudinal velocity, root mean squared error (RMSE) and direction error.

$Y^{<t+r>} = \{x_0, y_0, x_1, y_2, \dots, x_r, y_r\}$	$l_1^{<t+r>} = \sum_{i=1}^{r_0} \ p_{ego}^{<t+i>} - p_{dest}^{<t+r>}\ _2^2$	$l_2^{<t+r>} = \sum_{i=1}^{r_0} v_{\delta}^{<t+i>}$
$l_3^{<t+r>} = \sum_{i=1}^{r_0} v_f^{<t+i>} \in [v_{min}, v_{max}]$	$RMSE = \sum_{i=1}^{r_0} \frac{\sqrt{(\hat{p}_{ego} - p_{ego})^2}}{n}$	$SignLoss = \frac{\sum_{i=1}^n \sqrt{\frac{(\ \hat{x}_{ego}\ - \ x_{ego}\)^2}{n}}}{I(sign(\hat{x}) == sign(x))}$

FIGURE 20 OBJECTIVE FUNCTIONS USED

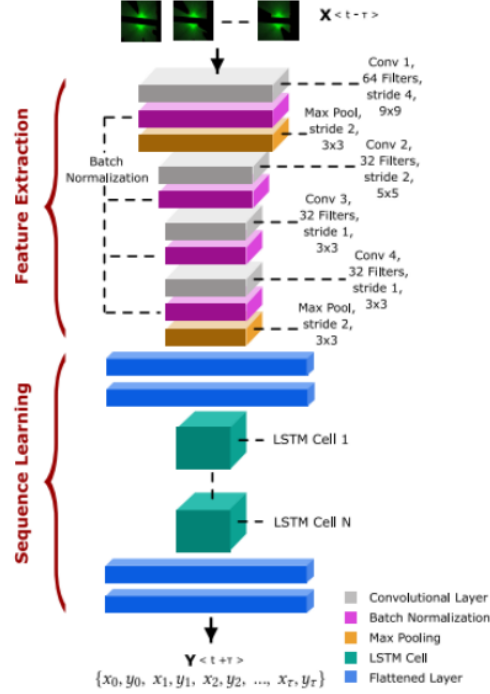


FIGURE 21 DIAGRAM OF NETWORK TOPOLOGY DEVELOPED USING NEURO-EVOLUTION

2.2.4.2 Data

Simulator: The GridSim simulator is used to generate input data consisting of sequenced occupancy grids, which are images captured from a highway driving scenario.

Data Composition: Each sequence consists of multiple images (128 x 128 pixels, RGB channels) representing the position of the ego vehicle over time.

Dataset: The dataset includes 30km of image data divided into training (60%), validation (20%), and test (20%) sets. A sliding window approach is used to increase the number of sequences.

Training and Testing: The data is preprocessed and split, with the training and validation sets shuffled before training. The test dataset is withheld for final evaluation.

2.2.4.2 Method Evaluation

Objective Evaluation: The impact of each objective function is analyzed through five experiments, each combining different sets of objectives to assess their effectiveness in trajectory prediction.

Metrics: Evaluation metrics include RMSE, distance feedback, lateral and longitudinal velocity, and direction error.

Results Analysis: The study examines the spread of predicted trajectories and identifies which objectives contribute positively or negatively to model performance. Experiments show that certain objectives, like distance feedback, can be detrimental, while others, like lateral velocity, can be beneficial.

metric	Experiment 1		Experiment 2		Experiment 3		Experiment 4		Experiment 5	
	mean	± std	mean	± std	mean	± std	mean	± std	mean	± std
RMSE _{val}	0.657413	0.032726	0.727919	0.057119	2.077965	0.351699	2.307763	0.234958	2.403594	0.201088
RMSE _{test}	1.063261	0.007949	1.049083	0.010070	1.605857	0.204658	1.734267	0.152781	1.746888	0.131945

FIGURE 22 : METRIC MEAN AND STANDARD DEVIATION (STD) FOR ALL RUNS OF A GIVEN EXPERIMENT

2.2.5 Cross Talk Neural Architecture Search for EEG Signal Classification

2.2.5.1 Strategy & Structure

(Duan et al., 2022) research's focuses on a CTNAS-EEG framework designed to enhance EEG signal recognition across different tasks. It follows a strategy to create only one prototype network, Meta-Net, which captures channel and time-wise information from the EEG signal. All eventualities provide full understanding about the data. It is specifically designed for an EEG signal search space and deals with issues like low SNR (signal-to-noise ratio) and high variability between subjects and tasks. Two constraints are applied to the network to ensure reliability in performance: network size reduction coupled with the use of many skip connections improves stability and training also avoiding overfitting. It specifically invokes a differentiable architecture search method known as DARTS (Liu et al., 2018) and solves the constrained optimization problem using Augmented Lagrangian techniques to balance performance and efficiency.

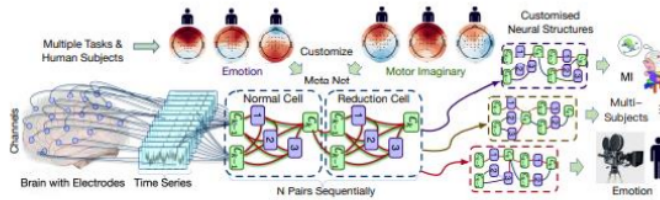


FIGURE 23 CTNAS-EEG META-NET



FIGURE 24 CTNAS-EEG SEARCH SPACE

2.2.5.2 Data

In this paper, two major datasets are used to evaluate the CTNAS-EEG framework. The first dataset is from the BCI Competition IV and consists of EEG and EOG signals of nine subjects who have

performed Motor Imagery tasks: left-hand, right-hand, feet, and tongue. All data was recorded at a frequency of 250 Hz with 22 EEG channels and 3 EOG channels. The second database was obtained from the SEED project, related to the recognition of emotion. This would amount to recordings from 15 subjects, each having three different sessions. All these sessions are purposed to catch emotions of happiness, neutrality, sadness, and fear. The recordings were done at a frequency of 1000 Hz using 62 EEG channels.

2.2.5.3 Method Evaluation

In this context, the evaluation of the CTNAS-EEG framework will involve multiple experiments with comparisons. There are mainly two kinds of performance metrics: accuracy and standard deviation, which measure model performance on mixed and subject-specific datasets. The experiments were conducted with PyTorch, and the data was pre-processed by normalization techniques as usual. Results show that on the Motor Imagery task, CTNAS-EEG achieved state-of-the-art performance, outperforming those manually designed models like EEG-TCNet. On Emotion Recognition tasks, CTNAS-EEG has also shown competitive accuracy for the effective unification of both the MI and Emotion tasks within one framework. Further, this study considers an analysis of cross-task searching, looking into the comparative effectiveness of selected operators across different tasks. The overall work is representing CTNAS-EEG for automatic design and optimization of neural network architecture in the recognition of EEG signals, ensuring high accuracy and efficiency with special regard to challenges specific to EEG data.

	Unified Weights						Subject-Specific Weights						Variable Structure	
	EEGNet*		EEG-TCNet		CTNAS-EEG		EEGNet*		EEG-TCNet		CTNAS-EEG		CTNAS-EEG	
	Acc.(%)	κ	Acc.(%)	κ	Acc.(%)	κ	Acc.(%)	κ	Acc.(%)	κ	Acc.(%)	κ	Acc.(%)	κ
S1	85.12	0.74	85.77	0.81	87.35	0.78	87.09	0.82	89.32	0.86	93.25	0.80	95.67	0.87
S2	61.05	0.32	65.02	0.53	72.41	0.58	62.38	0.52	72.44	0.63	80.52	0.71	81.35	0.76
S3	86.36	0.79	94.51	0.93	90.23	0.81	93.21	0.91	97.44	0.97	98.15	0.92	98.01	0.92
S4	65.77	0.67	64.91	0.53	67.22	0.58	74.57	0.58	75.87	0.68	79.65	0.69	81.20	0.67
S5	66.90	0.59	75.36	0.67	76.10	0.71	77.12	0.70	83.69	0.78	85.68	0.81	86.55	0.83
S6	54.01	0.41	61.40	0.49	62.60	0.44	62.10	0.51	70.69	0.61	76.24	0.60	77.35	0.69
S7	87.93	0.88	87.36	0.83	89.13	0.84	90.25	0.89	93.14	0.91	93.89	0.89	94.56	0.90
S8	77.34	0.81	83.76	0.78	79.21	0.76	86.57	0.85	86.71	0.82	89.27	0.84	90.05	0.84
S9	75.21	0.65	78.03	0.71	77.68	0.66	83.21	0.70	85.23	0.80	90.06	0.82	91.88	0.82
Avg.	73.29	0.651	77.36	0.698	78.10	0.684	79.72	0.72	83.84	0.784	87.41	0.786	88.52	0.809
Std.	12.06	0.186	11.59	0.155	10.02	0.131	11.50	0.156	9.20	0.125	7.39	0.102	7.28	0.088

FIGURE 25 CTNAS-EEG SUBJECT DEPENDENT PERFORMANCE

Chapter 3: Material and Methods

3.1 Materials

3.1.1 Data

The dataset required for this project to run will be collected in real-time using LIDAR

Sensors and optionally RGB Cameras for detection Tasks, while keeping in mind that RGB Cameras

seems to be vulnerable to mentioned vulnerabilities.

Optionally real-time data of non-invasive EEG will be collected and recorded for later analysis and training architecture to search the better preprocessing techniques to process this signal and Collect Related Data.

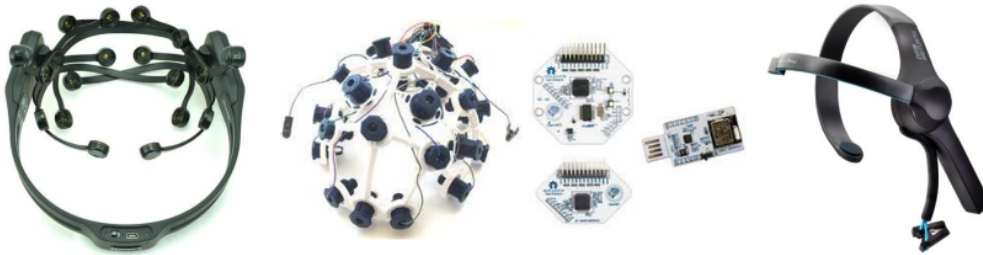


FIGURE 26 NON-INVASIVE EEG

Also, a Simulated Environment can be used to collect LIDAR Data intensively and train the model to even react to physical elements interaction that could happen rarely in real-life. So, finding a dataset of it is nearly impossible [a truck loaded with boxes happens to fall on a LIDAR mounted Car].

real-world self-driving datasets:

ATG4D:

large-scale Dataset of real-world car mounted with LIDAR, with over 1 million+ frames from 5000+ snippets. Also, these frames were collected from multiple cities in North America.

64-beam LIDAR were used. Also, the dataset contains HD Maps with challenging and cluttered scenes.

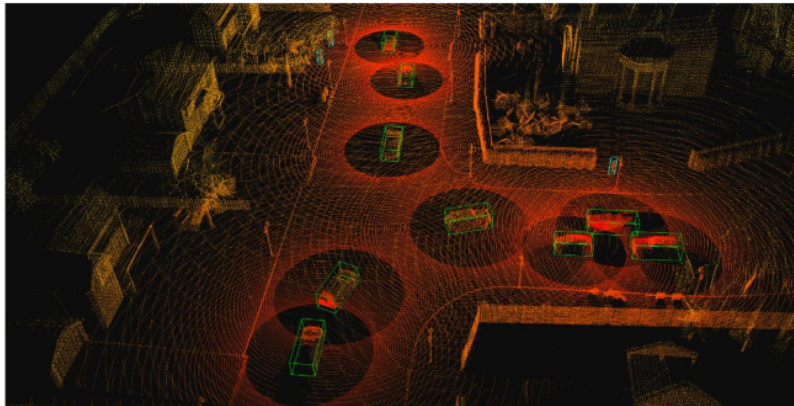


FIGURE 27 ATG4D

3.1.2 Tools

Conda/Anaconda: platform for efficiently developing Deep learning modules.

Python3: widely used language.

tensorflow: it provides High-Level implementation of widely used function in both Machine Learning and Deep Learning.

C/C++: maybe used in programming time sensitive module (e.g. EEG/LIDAR).

Open3d: mainly will be used to visualize charts in real-time efficiently.

Beamng.Tech: a commercial autonomous driving simulator used by Audi and other tech related companies, it offers a soft-body physics simulated environment, alongside many sensors used in real-life like LIDAR, Depth Cameras, Ultrasonic, IMU (input measuring unit) & RGB Cameras. Also, one of the best environments for training reinforcement models in real-life scenarios.(Maul et al., n.d.)

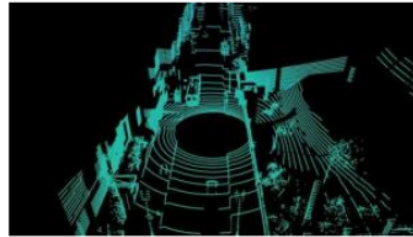


FIGURE 28 LIDAR Vs CAMERA DATA FROM BEAMNG

3.1.3 Environment

Local:

CPU: Intel Core i7 10th Generation, 10 Cores

GPU: GTX 1660 TI

Will be later in training process switch to cloud for better performance but Beamng

Could run up to 100 instances with average FPS.

3.2 Methods

3.2.1 System architecture Overview



FIGURE 29 SYSTEM ARCHITECTURE OVERVIEW

The proposed Advanced Driver Assistance System (ADAS) integrates multiple data sources and advanced algorithms to enhance driving safety. The system architecture includes data collection modules using LIDAR, RGB cameras, and EEG sensors, a data processing unit, a decision-making module using neuro-evolutionary algorithms, and a visualization component to provide real-time feedback to the driver.

3.2.2 Algorithm Description

As Shown previously in the related work section that Deep Learning are chosen to overcome the main challenges of perception and prediction of trajectories of objects using point cloud collected using LIDARs. Also, LSTMs and TCN as discussed by (Lin et al., 2022) in his research which had shown high optimistic results on prediction Human Decision with LSTM. Although LSTMs shows better performance than RNNs TCNs (Temporal Convolution Networks) which is an upgraded version of CNNs that work with making a conditional temporal convolution, in recent research (Ingolfsson et al., 2020) showed that TCNs can somehow outperform LSTMs in tasks that is mainly related to motor functions of humans.

As we dig even more, we will find 100s of research that will utilize Deep Learning approaches. But all of them rely mainly on fixed network structure which somehow is a trial-and-error solution. So, we took in mind developing a universal Solution to accomplish the task of manually designing the Deep Learning Network Structures, later we will discuss Neuro-Evolution as we hope that it will evolve better networks than that proposed in the related work section.

3.2.2.1 NEAT (Neuro-Evolution of Augmenting Topologies)

Training Deep Learning has two significant Drawback one of them is the use of vast amount of training samples which can be easily ignore if we are collecting data from multiple sources, The Second major drawback that we rely mainly on a fixed topology that is manually by the experimenter, which results in inefficient use of computational resources; Fortunately, Evolutionary Algorithms address the mentioned problems. These algorithms were development with the bio-inspiration of Theory of evolution by Charles Darwin, at the heart of any evolutionary algorithm relies on a two main genetic operation (mutation, crossover) which NEAT has took advantage of it to start a small initial network that later will evolve into More Complex ones. NEAT defines the genome which describes both the nodes and their connections which played a major role in evolving it. As, NEAT relies mainly on developing Simple Neurons it can be altered to accept more complex Architectures like (CNN, LSTMs, TCN and RNN). Also, a fitness function should be calculated upon finishing of crossover and mutation, those a rank could be given of each chromosome (network, candidate) and this fitness function can have a multi-objective like (network complexity, Time of Execution, etc.) which could fit well in our software as we need it to be time constrained.

The Genetic encoding Scheme of NEAT is designed to allow us easily matching between nodes during mating (crossover) process. While it holds crucial information about the network, on of the most important gene was the innovation number as it is used in the matching process.

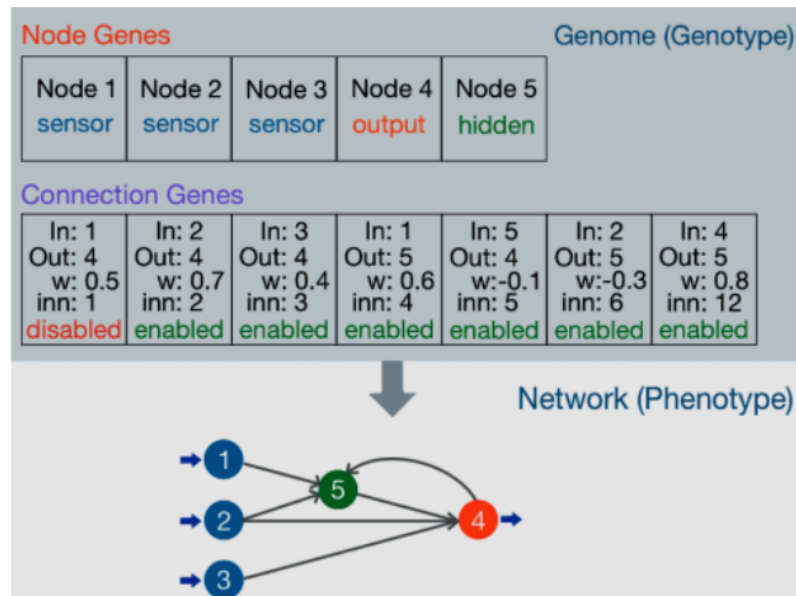


FIGURE 30 NEAT GENOME SCHEME

The mutation operation of NEAT can change connection's weight (strength) and the network's structure. The two main structural mutations are: Adding a new connection between nodes and adding a new node to the network as Shown in FIGURE 31.

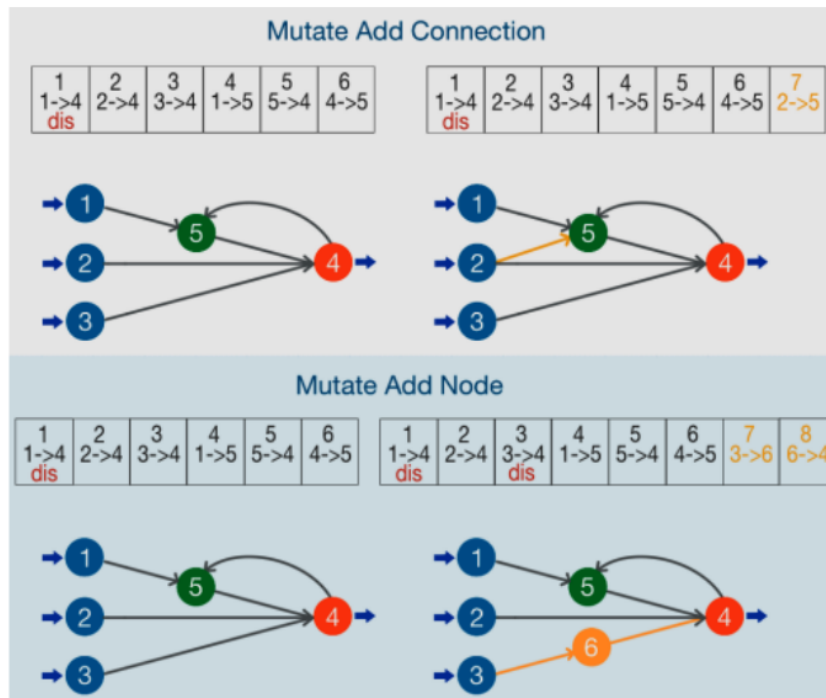


FIGURE 31 STRUCTURAL MUTATIONS OF NEAT

During crossover, genes with the same innovation number are considered matching and are inherited by the offspring from either parent. Genes that do not match (disjoint or excess genes) are handled specially, with the offspring inheriting these genes from the parent with the highest fitness. This mechanism ensures efficient gene recombination using linear genome encoding, avoiding complex topological analysis.

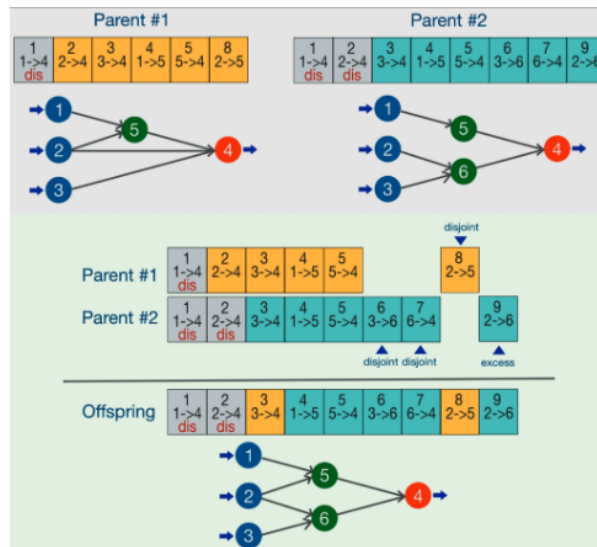


FIGURE 32 CROSSOVER OF NEAT

Speciation helps protect new and complex neural network designs. When new connections or nodes are added, these networks often start off with lower performance. Speciation groups similar networks together, so they only compete within their group, not against all networks. This gives new designs a chance to improve without being outcompeted by simpler ones, helping NEAT find better solutions over time.

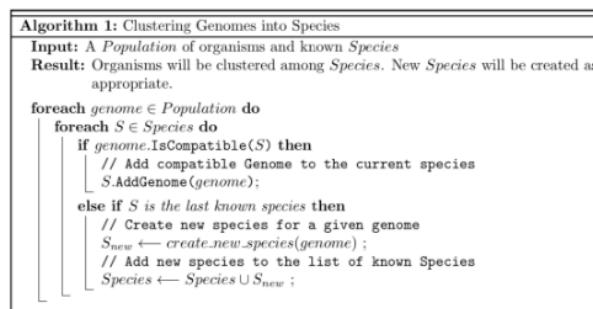


FIGURE 33 ALGORITHM OF SPECIATION

3.2.2.2 Differential Evolution (DE)

DE is a population-based stochastic evolutionary algorithm that aims to find global optimum solution for complex optimization problems (Neural Architecture Search in our case) (Das & Suganthan, 2011) , in the heart of this algorithm State-of-the-art stages are proposed that accomplish its task of converging to better solution (local minima) which are: initialization, mutation, crossover, and selection.

Upon finishing the initialization of population with random solutions, the mutation operation is applied to form a Donor Vectors, the most used mutation scheme is:

$$v_i^g = x_{best}^g + F * (x_{r1}^g - x_{r2}^g)$$

In this equation v_i^g denotes i^{th} donor vector of the target vector x_i at generation g ; x_{best}^g defines the best candidate solution of the g^{th} generation according to the selection process; (x_{r1}^g, x_{r2}^g) are two randomly mutually exclusive candidates chosen from the currently generation, at the End this mutation is governed by F a scaling factor $\in (0,1)$ defining the rate of mutation.

Although this defines a simple operation it had shown better performance (faster convergence to solution) than GA (genetic algorithm).

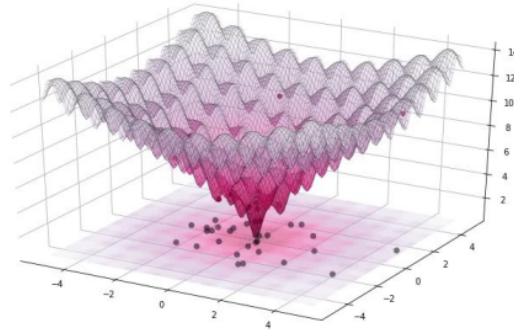


FIGURE 34 DE OPTIMIZING THE 2D ACKLEY FUNCTION

The main reason for such unique behavior is that the mutation uses byte/bit level operations, the following shows the common types of mutation operators. Also, the use of uniform Crossover ensures the vectors ability to exploit promising regions.

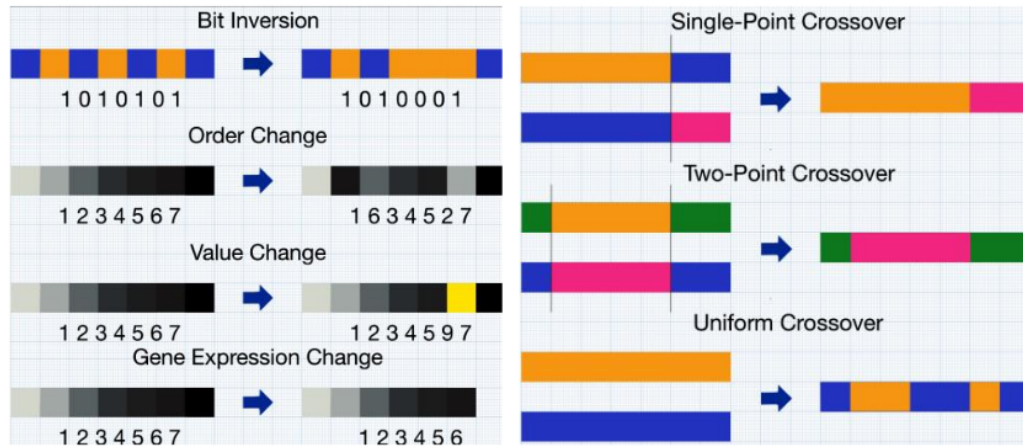


FIGURE 35 TYPES OF GENETIC OPERATORS

One of the main disadvantages we will try to solve is that the DE uses a fixed length vector to perform its genetic operators. So, a variable-length encoding technique will be developed to overcome this issue and in which we could develop much more diverse of Deep Learning Layers (LSTM, CNN) as they will crossover and mutate well. also, a method like genetic alignment proposed by (Sun et al., 2017a) could be useful to accomplish this task.



FIGURE 36 METHOD PROPOSED TO OVERCOME VARIABLE LENGTH CHROMOSOMES

3.2.2.2.3 Introducing Skip Connection with NEAT

Combining the strengths of Neuro-Evolution of Augmenting Topologies (NEAT) and Differential Evolution (DE) offers a robust approach for evolving and optimizing Graph Neural Network (GNN) architectures. NEAT is adept at evolving the topology of the GNN, allowing it to discover innovative and effective network structures by starting with simple configurations and progressively increasing their complexity through mutations and crossovers. This evolutionary process enables the network to adapt and improve, making it highly suitable for tasks that require sophisticated structural adaptations, such as those encountered in Advanced Driver Assistance Systems (ADAS).

while DE excels in fine-tuning the weights and hyperparameters of these architectures, enhancing their performance and efficiency. DE's optimization process ensures that the GNNs are not only structurally optimal but also finely tuned to perform specific tasks with high accuracy. By balancing exploration and exploitation, DE prevents premature convergence and allows the search for optimal solutions to be comprehensive and effective.

In the context of ADAS, this hybrid approach is particularly valuable. The task of predicting driver trajectory safety requires the integration and processing of diverse data sources, including LIDAR for environmental awareness and EEG for cognitive state analysis. NEAT's ability to evolve complex network topologies ensures that the GNN can effectively learn and represent the intricate relationships between these multimodal data inputs. Meanwhile, DE's fine-tuning capabilities ensure that the network parameters are optimized for real-time performance and accuracy, crucial for making timely and reliable safety decisions.

Thus, the combination of NEAT and DE creates a powerful method for developing GNNs that can handle the complexities of real-world driving environments, leading to advanced, intelligent systems that enhance road safety and driver assistance through predictive trajectory analysis and adaptive response

to cognitive and environmental inputs. This hybrid approach leverages the best of both worlds—NEAT’s structural evolution and DE’s parameter optimization—to achieve superior performance in ADAS applications.

Chapter 4: System Implementation

4.1 System Development

The implementation of the system is carried out using Python, with key libraries including TensorFlow/Keras for deep learning excluding the capabilities of utilizing Graph Neural Networks as mainly most of our work is using evolutionary techniques and most a randomly selection of architectures which will raise several build issues. As, pyTorch is much less forgiving while randomly changing its graph optimization of backpropagation. As, this behavior could be ignored if we choose to use the no-grad option while making the network. But for the sake of simplicity, we will use TensorFlow. For non-differentiable problem we recommend using non-gradient training methods like the Likelihood Ratio Method proposed by (Jiang et al., 2023) which had shown a nearly optimistic performance like the Gradient ones. Open3D is used for visualization specially 3D point cloud related data and could be later altered to achieve AR tasks, and BeamNG.tech for a comprehensive simulation platform that enables realistic driving scenarios and environment simulation rather than manually searching datasets for scenarios. BeamNG.tech is used to generate and test various driving situations, providing a controlled environment to collect training data for the ADAS. This data includes vehicle trajectories, collision scenarios, and driver behavior under different conditions, which are essential for training and evaluating the performance of the developed models. The codebase is structured into modules for data processing, model training, and evaluation, with key functions and classes designed to handle specific tasks such as LIDAR data processing, EEG signal analysis, and neuro-evolutionary optimization.

During the System Development we had tried lots of solutions and found that it is better to use cloud development Environment like (Colab / CodeSpace) which showed better performance and hijer

productivity as it comes shipped with compatible libraries to let us focus more on researching/development. Also, local runs had to include in order to run BeamNG.Tech as the BeamNG Team had supported us with a development license exposing its LUA internal engine so tasks like collecting point cloud could be conducted easily.

4.2 System Structure

Mainly the System will run in two phases Training and Deployment Phases. As our main goal is to develop a better ADAS our method introduces a Neural Architecture Search for the 3 Main models of system 1. Trajectory Analysis/Forecasting 2. Brain Signal Recognizer 3. Reinforcement Risk Estimator.

The Training Phase must be as generic as possible as the 3 main models require different Dataset types with 3 different approaches 1. Unsupervised 2. Supervised 3. Reinforcement Respectively. Also, each model will have his own search space while providing a heuristic of layer types which could be easily determined by related work manually crafted architectures. For Example:

Trajectory Related → Conv + Pooling + LSTM (Stapleton et al., 2022) / TCN (Li et al., 2023)

Brain Signal Forecasting → LSTM + Dense (Lin et al., 2022)/ TCN (Ingolfsson et al., 2020) / Conv + pooling + Dense(Duan et al., 2022)

Or using surveys like the one made by (Bharilya & Kumar, 2023)on the field of trajectory analysis.

The Deployment phase will require us to deploy the best candidates of our NAS (Neural Architecture Search) and then Training using Gradient method if needed to improve it performance even well and then we will have to ensure that this topology will survive in much more intensive Scenarios rather than only leaving it without any further improvement.

4.2.1 System Overview

In this section we will describe the proposed system Advanced Driver Assistance System (ADAS) using neuro-evolutionary trajectory prediction and brain analysis with LIDAR, EEG, and AR. The system comprises several key stages:

4.2.1.1 Data preparation

Mainly the two Data Sources which we will collect our data from are BeamNG.Tech simulated LIDAR or real-time LIDAR. Also, EEG headset.

```
# file selection view | Run Normal Help | Search
Welcome | bodeq.pyb X
+ Code + Markdown | ▶ Run All | Restart | Clear All Outputs | Variables | Outline ...
+ Code + F Markdown

import random
from time import sleep

from beamngpy import BeamNGPy, Scenario, Vehicle, set_up_simple_logging
from beamngpy.sensors import Lidar
HOM_HOST = "70\\Victorov-12ch-on-11.2.it\\*"
HWI_IP = "70\\Victorov-12ch-on-11.2.it\\grafik"

random.seed(198)
set_up_simple_logging()
beamng = BeamNGPy("localhost", 64256, beam-HWI_IPW, user=HOM_USER)
bg = beamng.open(launch=True)

scenario = bg.scenarios.get_scenarios()
# list(scenario.keys())

dist_keys(["jungle_rack_island", "automation_test_track", "Urban", "Industrial", "grindap_v2", "Italy", "johnson_valley", "east_coast_usa", "west_coast_usa", "driver_training", "derby", "small_island", "Cliff"])

driver_scenario = scenario["driver_training"]
print(f"There are {len(driver_scenario)} scenarios that can be loaded.")
driver_scenario

# There are 2 scenarios that can be loaded.

[{"scenario": "level-driver_training", "name": "beginning", "path": "/levels/driver_training/scenarios/beginning_jun.jun"}, {"scenario": "level-driver_training", "name": "beginning", "path": "/gameplay/missions/campaigns/driver_training/m1-starting/info_jun.jun"}]

scenario = Scenario("driver_training", "beginning", description="Grad 1 test scenario" # path="/levels/west_coast_usa/scenarios/l10m_demo_jun")
vehicle = vehicle("ego_vehicle", model="ek800", license="RUP", colour="red")

scenario.add_vehicle(vehicle, pos=[200.0, 0, 0.5228])
first_pos=(0.0, 0.5228254, 0.5228795)

scenario.make(bg)

bg.settings.set_deterministic(0) # Set simulator to 60Hz temporal resolution
bg.scenario.load(scenario)
# bg.ui.hide_tool()
bg.scenario.start()

2024-03-11 21:40:52,273 INFO | beamngpy.BeamNGPy | loaded ego_...
2024-03-11 21:40:53,340 INFO | beamngpy.Vehicle | (vehicle ego_vehicle connected to simulation.
2024-03-11 21:40:53,341 INFO | beamngpy.BeamNGPy | Attempting to connect to vehicle ego_vehicle
2024-03-11 21:40:55,066 INFO | beamngpy.BeamNGPy | Successfully connected to beamng.tech
2024-03-11 21:40:55,066 INFO | beamngpy.BeamNGPy | Successfully connected to vehicle ego_vehicle.
2024-03-11 21:40:55,063 INFO | beamngpy.Scenario | Connected to scenario: beginning
2024-03-11 21:40:55,064 INFO | beamngpy.Vehicle | Starting scenario.

lidar = Lidar('lidar1', beamng, vehicle, requested_update_time=0.01, is_using_shared_memory=False) # Send data through us socket instead.
vehicle.ai.set_mode('son')
print(f'Ordering around, polling the LIDAR sensor every 1 seconds...')
for i in range(100):
    sleep(1)
    readings_data = lidar.poll() # Fetch the latest readings from the sensor, over either shared memory or the us socket.
    # print(f'Lidar Poll cloud data after ~{i * 0.01} seconds': readings_data['pointcloud'][:10]) # The first 10 points from LIDAR poll cloud data.
    # print(f'Readings data after ~{i * 0.01} seconds': readings_data['colour'][:10]) # The colour data (corresponds to each point).
    # visualize_readings_data('pointcloud')
    # visualize_readings_data('readings_data')
    lidar.remove()
beamng.close()
```

FIGURE 37 LOADING LIDAR DATA FROM BEAMNG.TECH

4.2.1.2 Model Training

For the sake of simplicity, we will evolve our network on widely use dataset comparing previous work on it to have an easily comparison and evaluation of our neuro-evolutionary method

4.2.1.2.1 INTIALIZATIONS

```
Input:  $N$ , population size;  $g_{max}$ , maximum generation number;  $CR$ , crossover rate;  $F$ , scaling factor;  $D_{train}$ , training dataset;  $D_{valid}$ , validation dataset;  $D_{test}$ , test dataset.  
Output: Return best found individual  $x_{best}$  and its test error  
1  $P = \{x_1, \dots, x_N\} \leftarrow \text{population\_initialization}(N)$   
   //Initialize population  
2 for  $i = 1$  to  $N$  do  
   3  $x_i \leftarrow \text{random\_configuration}(\text{layer\_type}, \text{hyper-parameters}, l_{max})$   
   //randomly initialize Conv, Pool and FC layer along with their hyper-parameter  
   and maximum length of individual  
   4  $f(x_i) \leftarrow \text{compute\_fitness}(x_i, D_{train}, D_{valid})$   
   //Fitness evaluation  
   5  $fitness(i) = f(x_i)$   
6 end  
7  $x_{best} \leftarrow \min(fitness)$ ; //best individual  
8 While  $g \leq g_{max}$  do  
   9 for  $i = 1$  to  $N$  do  
    10  $v_i^g \leftarrow \text{compute\_mutation}(x_i^g, x_{c1}, x_{c2}, x_{best}, F)$   
    //Donor vector generation using mutation operation  
    11  $u_i^g \leftarrow \text{compute\_crossover}(x_i^g, v_i^g, \delta, CR)$   
    //Trial vector generation using crossover operation  
    12  $fitness(i) = f(u_i^g)$   
    //Fitness evaluation of trial vector  
    13  $x_i^{g+1} \leftarrow \text{compute\_selection}(f(x_i^g), f(u_i^g))$  //Selection for next generation  
    14 end  
    15  $g \leftarrow g + 1$   
16 end  
17  $x_{best} \leftarrow \min(fitness)$   
   //Final best CNN architecture  
18 for  $e_1 \leq \text{best\_trainepoch}$  do  
   19  $x_{best}^{train} \leftarrow \text{train}(x_{best}, D_{train}, \text{parameters})$   
   //  $x_{best}$  train with training data and parameters  
20 end  
21 for  $e_2 \leq \text{best\_testepoch}$  do  
   22  $f(x_{best}) \leftarrow \text{test}(x_{best}^{train}, D_{test})$   
23 end  
24 Return  $x_{best}$  and its test error
```

FIGURE 38 INITIALIZATION PSEUDOCODE

4.2.1.2.1 MUTATION AND Crossover

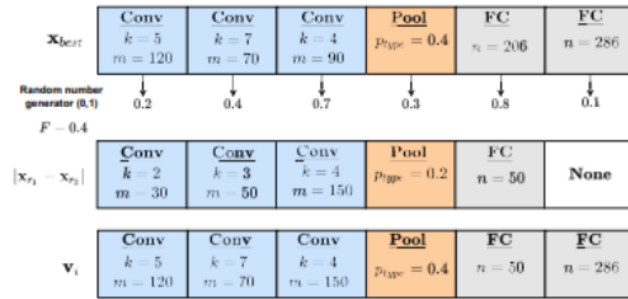


FIGURE 39 DONOR VECTOR GENERATING USING MUTATION OPERATION

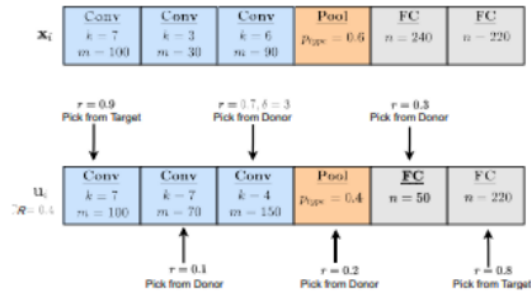


FIGURE 40 TRIAL VECTOR GENERATING USING Crossover OPERATION

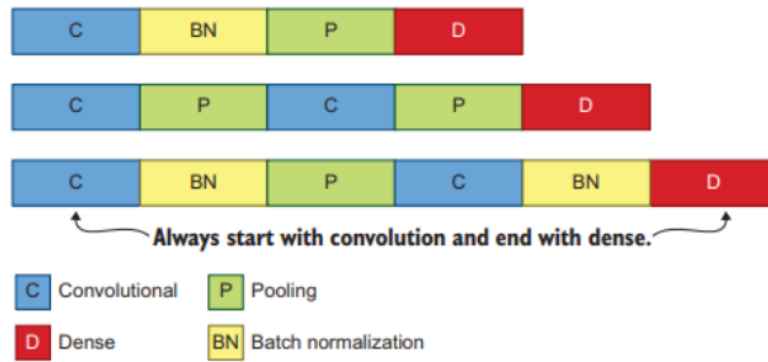


FIGURE 41 VARIABLE ENCODING LENGTH GENE

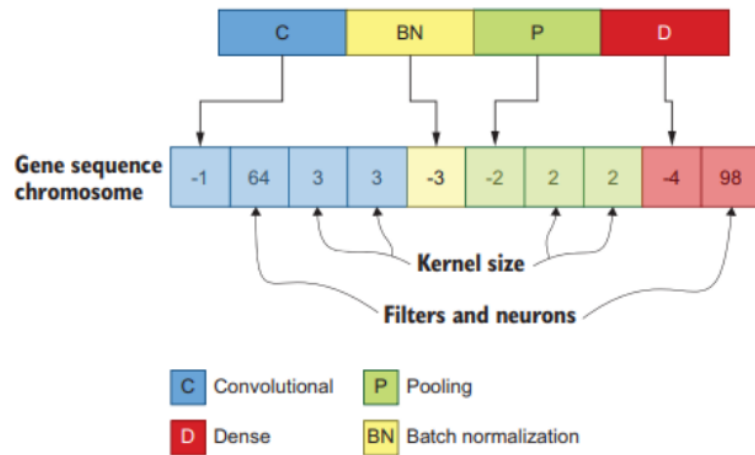


FIGURE 42 GENE ENCODING THE CNN ARCHITECTURE

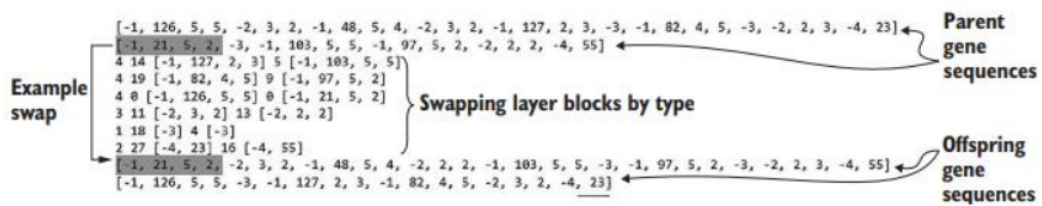


FIGURE 43 EXAMINING CROSSOVER

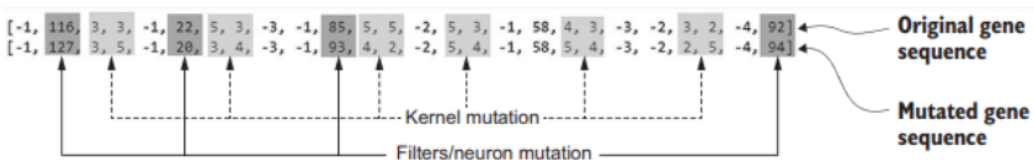


FIGURE 44 EXAMINING MUTATION

4.2.1.3 Prediction and Feedback

After training the models to find the best architecture we will use it to have a functional system.

Models and solutions that we mainly rely on in our system.

Trajectory Prediction: Use the trained model to predict the future trajectory of the vehicle.

Brain Signal Analysis: Analyze EEG data to predict the driver's intended actions.

AR Feedback: Display predictions and warnings to the driver using AR technologies.

4.2.2 Source Code

```

In [1]: !@title Install Packages
        !pip install livelossplot --quiet
        !pip install deap --quiet

135.4/135.4 KB 257.8 KB/s eta 0:00:00

In [2]: !@title Imports
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import math
import time
import random

import matplotlib.pyplot as plt
from livelossplot import PlotLossesKeras, PlotLosses
from IPython.display import clear_output

#DEAP
from deap import algorithms
from deap import base
from deap import benchmarks
from deap import creator
from deap import tools

In [3]: !@title Load Data
dataset = datasets.fashion_mnist
(x_train, y_train), (x_test, y_test) = dataset.load_data()

# normalize and reshape data
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1).astype("float32") / 255.0
x_test = x_test.reshape(x_test.shape[0], 28, 28, 1).astype("float32") / 255.0

x_train = x_train[:10000]
y_train = y_train[:10000]
x_test = x_test[:1000]
y_test = y_test[:1000]

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 [=====] - 86 B/s/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
36421888/36421888 [=====] - 25 B/s/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 [=====] - 86 B/s/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
442182/442182 [=====] - 16 B/s/step

In [4]: class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                        'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

In [5]: def plot_data(num_images, images, labels):
        grid = math.ceil(math.sqrt(num_images))
        plt.figure(figsize=(grid*2, grid*2))
        for i in range(num_images):
            plt.subplot(grid, grid, i+1)
            plt.xticks([])
            plt.yticks([])
            plt.grid(False)
            plt.imshow(images[i].reshape(28, 28))
            plt.xlabel(class_names[labels[i]])
            plt.show()

        plot_data(25, x_train, y_train)

```



```
In [6]: @title Constants
max_layers = 5
max_neurons = 128
min_neurons = 16
max_kernel = 5
min_kernel = 2
max_pool = 3
min_pool = 2

CONV_LAYER = -1
CONV_LAYER_LEN = 4
POOLING_LAYER = -2
POOLING_LAYER_LEN = 3
BN_LAYER = -3
BN_LAYER_LEN = 1
DENSE_LAYER = -4
DENSE_LAYER_LEN = 2
```

```
In [7]: @title Encoding scheme
def generate_neurons():
    return random.randint(min_neurons, max_neurons)

def generate_kernel():
    part = []
    part.append(random.randint(min_kernel, max_kernel))
    part.append(random.randint(min_kernel, max_kernel))
    return part

def generate_bn_layer():
    part = [BN_LAYER]
    return part

def generate_pooling_layer():
    part = [POOLING_LAYER]
    part.append(random.randint(min_pool, max_pool))
    part.append(random.randint(min_pool, max_pool))
    return part

def generate_dense_layer():
    part = [DENSE_LAYER]
    part.append(generate_neurons())
    return part

def generate_conv_layer():
    part = [CONV_LAYER]
    part.append(generate_neurons())
    part.extend(generate_kernel())
    return part

def create_offspring():
    ind = []
    for i in range(max_layers):
        if i==0: #first layer always convolutational
            ind.extend(generate_conv_layer())
        elif random.uniform(0,1)<.5:
            #add convolution layer
            ind.extend(generate_conv_layer())
        elif random.uniform(0,1)<.5:
            #add batchnormalization
            ind.extend(generate_bn_layer())
        elif random.uniform(0,1)<.5:
            #add max pooling layer
            ind.extend(generate_pooling_layer())
        else:
            ind.extend(generate_dense_layer())
    return ind

individual = create_offspring()
print(individual)

[-1, 69, 4, 3, -1, 27, 4, 3, -3, -4, 115]
```

```

In [8]: def build_model(individual):
        model = models.Sequential()
        ll = len(individual)
        i = 0
        while i < ll:
            if individual[i] == CONV_LAYER:
                n = individual[i+1]
                k = (individual[i+2], individual[i+3])
                i += CONV_LAYER_LEN
                if i == 0: #first Layer, add input shape
                    model.add(layers.Conv2D(n, k, activation='relu', padding='same', input_shape=(28, 28, 1)))
                else:
                    model.add(layers.Conv2D(n, k, activation='relu', padding='same'))
            elif individual[i] == POOLING_LAYER: #add pooling layer
                k = k = (individual[i+1], individual[i+2])
                i += POOLING_LAYER_LEN
                model.add(layers.MaxPooling2D(k, padding='same'))
            elif individual[i] == BN_LAYER: #add batchnormal layer
                model.add(layers.BatchNormalization())
                i += 1
            elif individual[i] == DENSE_LAYER: #add dense layer
                model.add(layers.Flatten())
                model.add(layers.Dense(individual[i+1], activation='relu'))
                i += 2
            else:
                break
        model.add(layers.Dense(10))
        return model

model = build_model(individual)

In [9]: def get_layers(ind, layer_type):
        return [a for a in range(len(ind)) if ind[a] == layer_type]

def swap(ind1, iv1, ind2, iv2, ll):
    ch1 = ind1[iv1:iv1+ll]
    ch2 = ind2[iv2:iv2+ll]
    ind1[iv1:iv1+ll] = ch2
    ind2[iv2:iv2+ll] = ch1
    return ind1, ind2

def swap_layers(ind1, ind2, layer_type, layer_len):
    c1, c2 = get_layers(ind1, layer_type), get_layers(ind2, layer_type)
    min_c = min(len(c1), len(c2))
    for i in range(min_c):
        if random.random() < 1:
            i1 = random.randint(0, len(c1)-1)
            i2 = random.randint(0, len(c2)-1)
            iv1 = c1.pop(i1)
            iv2 = c2.pop(i2)
            ind1, ind2 = swap(ind1, iv1, ind2, iv2, layer_len)
    return ind1, ind2

def crossover(ind1, ind2):
    ind1, ind2 = swap_layers(ind1, ind2, CONV_LAYER, CONV_LAYER_LEN)
    ind1, ind2 = swap_layers(ind1, ind2, POOLING_LAYER, POOLING_LAYER_LEN)
    ind1, ind2 = swap_layers(ind1, ind2, BN_LAYER, BN_LAYER_LEN)
    ind1, ind2 = swap_layers(ind1, ind2, DENSE_LAYER, DENSE_LAYER_LEN)
    return ind1, ind2

ind1 = create_offspring()
ind2 = create_offspring()
print(ind1)
print(ind2)

ind1, ind2 = crossover(ind1, ind2)
print(ind1)
print(ind2)

[-1, 111, 2, 2, -1, 85, 3, 4, -3, -2, 3, 2, -4, 104]
[-1, 62, 3, 5, -1, 112, 4, 4, -3, -1, 63, 5, 2, -3, -2, 3, 2, -1, 54, 2, 2, -2, 3, 2, -4, 76]
[-1, 54, 2, 2, -1, 62, 3, 5, -3, -2, 3, 2, -4, 76]
[-1, 85, 3, 4, -1, 112, 4, 4, -3, -1, 63, 5, 2, -3, -2, 3, 2, -1, 111, 2, 2, -2, 3, 2, -4, 104]

```

```

In [10]: @title Mutation
def mutate(part, layer_type):
    if layer_type == CONV_LAYER and len(part)==CONV_LAYER_LEN:
        part[1] = int(part[1] * random.uniform(.9, 1.1))
        part[2] = random.randint(min_kernel, max_kernel)
        part[3] = random.randint(min_kernel, max_kernel)
    elif layer_type == POOLING_LAYER and len(part)==POOLING_LAYER_LEN:
        part[1] = random.randint(min_kernel, max_kernel)
        part[2] = random.randint(min_kernel, max_kernel)
    elif layer_type == DENSE_LAYER and len(part)==DENSE_LAYER_LEN:
        part[1] = int(part[1] * random.uniform(.9, 1.1))
    else:
        error = f"mutate ERROR {part}"
        raise Exception(error)
    return part

def mutate_layers(ind, layer_type, layer_len):
    layers = get_layers(ind1, layer_type)
    for layer in layers:
        if random.random() < 1:
            try:
                ind[layer:layer+layer_len] = mutate(ind[layer:layer+layer_len], layer_type)
            except:
                print(layers)
    return ind

print(ind1)

def mutation(ind):
    if len(ind) > CONV_LAYER_LEN: #only mutate conv individuals
        ind = mutate_layers(ind, CONV_LAYER, CONV_LAYER_LEN)
        ind = mutate_layers(ind, DENSE_LAYER, DENSE_LAYER_LEN)
        ind = mutate_layers(ind, POOLING_LAYER, POOLING_LAYER_LEN)
    return ind,

ind, = mutation(ind1)
print(ind)

[-1, 54, 2, 2, -1, 62, 3, 5, -3, -2, 3, 2, -4, 76]
[-1, 56, 5, 2, -1, 65, 4, 4, -3, -2, 2, 5, -4, 70]

```

```

In [11]: @title Build, Compile and Train the model
def build_compile_train(ind, epochs=3):
    model = build_model(ind)

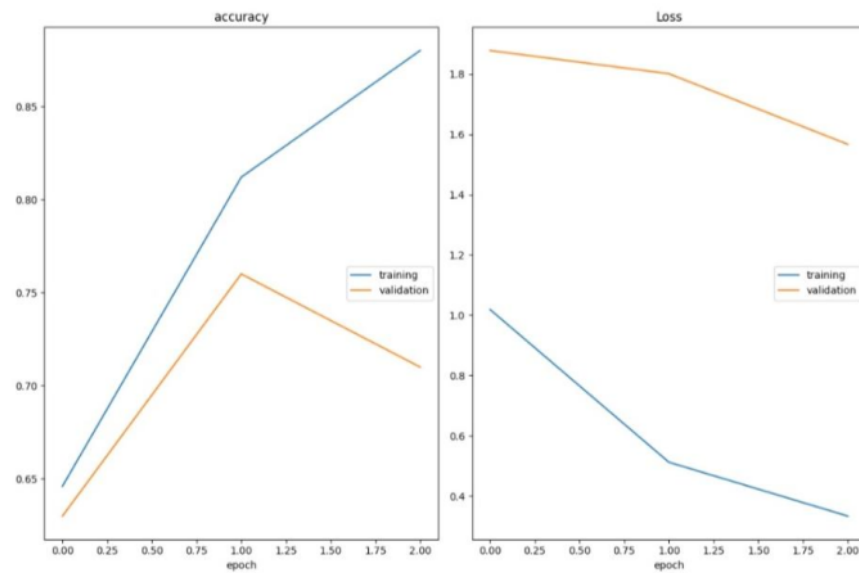
    model.compile(optimizer='adam',
                  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                  metrics=['accuracy'])

    history = model.fit(x_train, y_train, epochs=epochs,
                        validation_data=(x_test, y_test),
                        callbacks=[PlotLossesKeras()],
                        verbose=0)

    model.summary()
    model.evaluate(x_test, y_test)

build_compile_train(ind)

```



```

accuracy
  training (min: 0.646, max: 0.880, cur: 0.880)
  validation (min: 0.630, max: 0.760, cur: 0.710)
Loss
  training (min: 0.333, max: 1.019, cur: 0.333)
  validation (min: 1.567, max: 1.878, cur: 1.567)
Model: "sequential_1"

```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 28, 28, 56)	616
conv2d_3 (Conv2D)	(None, 28, 28, 65)	58305
batch_normalization_1 (Batch Normalization)	(None, 28, 28, 65)	260
max_pooling2d (MaxPooling2D)	(None, 14, 6, 65)	0
flatten_1 (Flatten)	(None, 5460)	0
dense_2 (Dense)	(None, 70)	382270
dense_3 (Dense)	(None, 10)	710

```

Total params: 442161 (1.69 MB)
Trainable params: 442031 (1.69 MB)
Non-trainable params: 130 (520.00 Byte)

```

```

4/4 [=====] - 0s 7ms/step - loss: 1.5669 - accuracy: 0.7100

```

Evolutionary CNN

```

In [12]: #@title Setting up the Creator
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

In [13]: #@title Create Individual and Population
toolbox = base.Toolbox()
toolbox.register("network", create_offspring)
toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.network)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("select", tools.selTournament, tournsize=5)

In [14]: #@title Register Crossover and Mutation
toolbox.register("mate", crossover)
toolbox.register("mutate", mutation)

In [15]: #@title Register Evaluation
def clamp(num, min_value, max_value):
    return max(min(num, max_value), min_value)

def compile_train(model):
    model.compile(optimizer='adam',
                  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                  metrics=['accuracy'])
    model.fit(x_train, y_train, epochs=3,
              verbose=0)
    return model

def evaluate(individual):
    try:
        model = build_model(individual)
        model = compile_train(model)
        print('.', end='')
        return 1/clamp(model.evaluate(x_test, y_test, verbose=0)[1], .00001, 1),
    except:
        return 1/.5,

toolbox.register("evaluate", evaluate)

In [16]: #@title Optimize the Weights (run: "auto")
MU = 10 #@param {type:"slider", min:5, max:1000, step:5}
NGEN = 5 #@param {type:"slider", min:5, max:100, step:1}
RGEN = 1 #@param {type:"slider", min:1, max:100, step:1}
CXPB = .6
MUTPB = .3

random.seed(64)

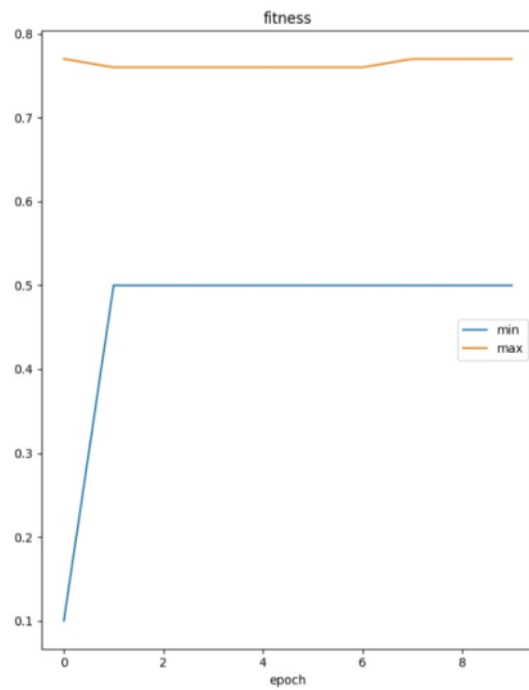
pop = toolbox.population(n=MU)
hof = tools.HallOfFame(1)
stats = tools.Statistics(lambda ind: ind.fitness.values)
stats.register("avg", np.mean)
stats.register("std", np.std)
stats.register("min", np.min)
stats.register("max", np.max)

In [17]: best = None
groups = { "fitness" : ("min", "max")}
plotlosses = PlotLosses(groups=groups)

for g in range(NGEN):
    pop, logbook = algorithms.eaSimple(pop, toolbox,
                                      cxpb=CXPB, mutpb=MUTPB, ngen=RGEN, stats=stats, halloffame=hof, verbose=False)
    best = hof[0]

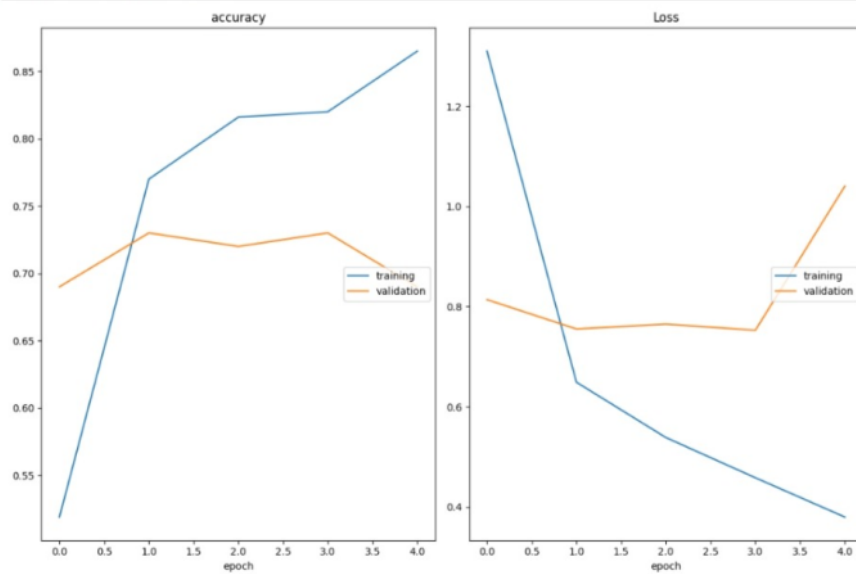
    print(f"Gen {(g+1)*RGEN}")
    for l in logbook:
        plotlosses.update({'min': 1/l["max"], 'max': 1/l["min"]})
    plotlosses.send() # draw, update logs, etc

```

fitness
 min (min: 0.100, max: 0.500, cur: 0.500)
 max (min: 0.760, max: 0.770, cur: 0.770)

In [18]: build_compile_train(best, epochs=5)



```

accuracy
  training      (min: 0.519, max: 0.865, cur: 0.865)
  validation    (min: 0.690, max: 0.730, cur: 0.690)
Loss
  training      (min: 0.380, max: 1.310, cur: 0.380)
  validation    (min: 0.753, max: 1.040, cur: 1.040)
Model: "sequential_48"

Layer (type)                 Output Shape                 Param #
-----
conv2d_123 (Conv2D)          (None, 28, 28, 49)          784
conv2d_124 (Conv2D)          (None, 28, 28, 39)          15327
max_pooling2d_52 (MaxPooli   (None, 14, 14, 39)          0
ng2D)
conv2d_125 (Conv2D)          (None, 14, 14, 66)          41250
flatten_34 (Flatten)         (None, 12936)                0
dense_82 (Dense)             (None, 69)                   892653
dense_83 (Dense)             (None, 10)                   700
-----
Total params: 950714 (3.63 MB)
Trainable params: 950714 (3.63 MB)
Non-trainable params: 0 (0.00 Byte)

4/4 [-----] - 0s 5ms/step - loss: 1.0404 - accuracy: 0.6900

```

FIGURE 45 FULL NEURAL ARCHITECTURE SEARCH IMPLEMENTATION

Source Code: <https://github.com/kuhl33d/NEAT-DE-NAS>

Input: Multiple LiDAR sweeps compensated by ego-motion L and raster map M : $\Omega = \{L, M\}$. Number of message passing steps of GNN K .

Output: Detections (confidences and bounding boxes) $D = \{c_0, b_0, c_1, b_1, \dots, c_N, b_N\}$. Prediction of statistics $O^K = \{o_0^K, o_1^K, \dots, o_N^K\}$

```

1:  $\{c_0, b_0, c_1, b_1, \dots, c_N, b_N\} \leftarrow \text{Detector}(\Omega)$  ▷ Includes NMS and confidence thresholding
2: for  $i = 1, \dots, N$  do ▷ Compute actor initial features and trajectories independently
3:    $r_i \leftarrow \text{RRoiAlign}(b_i)$ 
4:    $h_i^0 \leftarrow \text{MaxPooling}(\text{CNN}(r_i))$ 
5:    $o_i^0 \leftarrow \text{MLP}(h_i^0)$ 
6: Construct  $G = (V, E)$  a graph from detections
7: Compute pairwise coordinate transformations  $\mathcal{T}_{u,v}, \forall (u, v) \in E$ 
8: for  $k = 1, \dots, K$  do ▷ Loop over graph propagations
9:   for  $(u, v) \in E$  do ▷ Compute message for every edge in the graph
10:     $m_{u \rightarrow v}^{(k)} = \mathcal{E}^{(k)}(h_u^{k-1}, h_v^{k-1}, \mathcal{T}_{u,v}(o_u^{(k-1)}), o_v^{(k-1)}, b_u, b_v)$ 
11:   for  $v \in V$  do ▷ Update node states
12:     $a_v^{(k)} = \mathcal{A}^{(k)}(\{m_{u \rightarrow v}^{(k)} : u \in \mathbf{N}(v)\})$  ▷ Aggregate messages from neighbors
13:     $h_v^{(k)} = \mathcal{U}^{(k)}(h_v^{(k-1)}, a_v^{(k)})$  ▷ Update the hidden state
14:     $o_v^{(k)} = \mathcal{O}^{(k)}(h_v^{(k)})$  ▷ Update the output state (probabilistic trajectory)
15: return  $D, O^K$ 

```

FIGURE 46 PSEUDO CODE FOR EVOLVING A TRAJECTORY PREDICTION ON POINT CLOUD

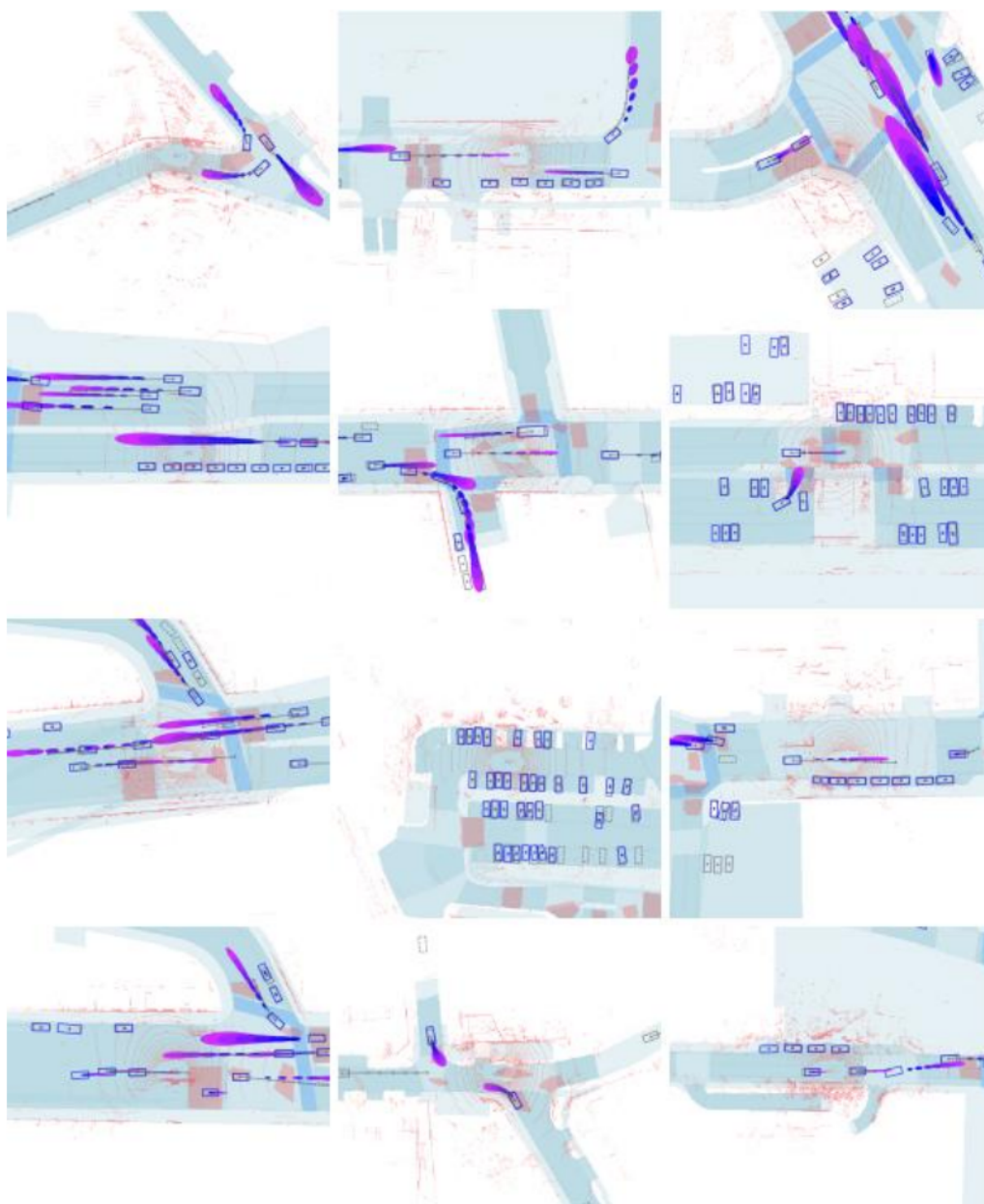


FIGURE 47 OUTPUT WHEN FUSING WITH BEAMNG.TECH SIMULATED LIDAR

4.3 SYSTEM RUNNING

In this section, we will demonstrate the input-output for each component in our final system.

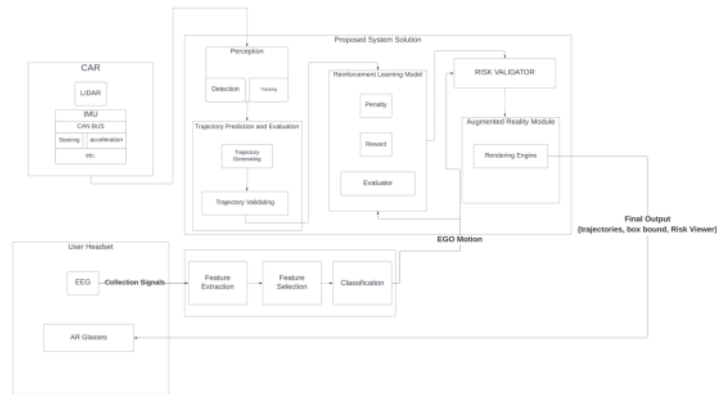


FIGURE 29 SYSTEM ARCHITECTURE OVERVIEW

4.3.1 Component A: Car Components

Input:

The car is equipped with various sensors and modules that provide essential data for the Advanced Driver Assistance System (ADAS):

LIDAR: Provides a detailed point cloud representation of the environment, crucial for 3D object detection and tracking.

IMU (Inertial Measurement Unit): Measures the car's acceleration, rotation, and other motion parameters.

CAN Bus: Collects data from various car components, including steering, acceleration, and braking systems.

Output:

These sensors continuously gather data, which is transmitted to the system for processing and analysis.

4.3.2 Component B: System Components

4.3.2.1 Perception Unit

Input:

LIDAR data.

Output:

The perception unit utilizes the Fast and Furious 3D object detection method to process LIDAR data and detect objects in real-time. (Zeng et al., 2021)

Detection: Identifies objects in the 3D point cloud data.

Tracking: Monitors the movement of detected objects over time.

Result: Positions and velocities of surrounding objects, which are essential for trajectory prediction and risk assessment.

4.3.2.2 Trajectory Prediction and Evaluation

Input:

Positions and velocities of surrounding objects.

Output:

This module uses techniques inspired by the End-to-end Interpretable Neural Motion Planner to predict and evaluate possible trajectories for the car.

Trajectory Generating: Produces potential future paths based on current object positions and motion trends.

Trajectory Validating: Assesses the feasibility and safety of each generated trajectory, ensuring they are free from collisions and other risks.

4.3.2.3 reinforcement Learning Model

Input:

Sensors inputs from the and the perception unit.

Output:

A reinforcement learning model, enhanced with neuro-evolutionary techniques, optimizes decision-making based on sensor inputs.

Evaluator: Assesses the safety and efficiency of different driving actions.

Penalty and Reward System: Guides the learning process by penalizing unsafe actions and rewarding safe, efficient ones.

Result: Safest trajectory for the car.

4.3.2.4 Risk Validator

Input:

Proposed Trajectories

Output:

The risk validator assesses potential hazards and validates the safety of proposed trajectories, working closely with the reinforcement learning model.

Result:

Selected paths minimize the risk of accidents.

4.3.2.5 Augmented Reality

Input:

Data from the perception unit and trajectory prediction.

Output:

The augmented reality module integrates with the user's AR glasses, providing a visual overlay of the analyzed data and detected objects.

Rendering Engine: Visualizes trajectories, object boundaries, and potential hazards in real-time.

Result: Ensures the driver is well-informed of the surroundings and suggested actions.

4.3.3 Component C: User Components

4.3.3.1 EEG Headset

Input:

Driver's brain signals.

Output:

The EEG headset continuously monitors the driver's brain signals, detecting their intent and readiness to perform actions.

Feature Extraction: Identifies relevant features from raw EEG signals.

Feature Selection: Chooses the most informative features for further processing.

Classification: Classifies the driver's intent using a pre-trained neuro-evolutionary model.

Result: Enables the system to anticipate and assist with driving actions.

4.3.3.2 AR Glasses

Input:

Real-time information from the augmented reality module.

Output:

The AR glasses display real-time information, including:

Trajectory Suggestions: Shows the safest paths suggested by the system.

Hazard Warnings: Highlights potential risks and hazards on the road.

Object Boundaries: Visualizes detected objects and their movements.

4.3.4 Final Output

Input:

Comprehensive data from all system components.

Output:

The final output of the system is a comprehensive set of trajectory suggestions, object boundaries, and risk assessments presented to the driver through the AR glasses. This

output ensures the driver can make informed decisions, enhancing overall safety and driving experience.

Chapter 5 Results and Evaluation

5.1 Testing Methodology

For the sake of simplicity research results conducted had been done over MNIST Fashion 10 to evaluate our neuro-Evolutionary approach

5.2 Results

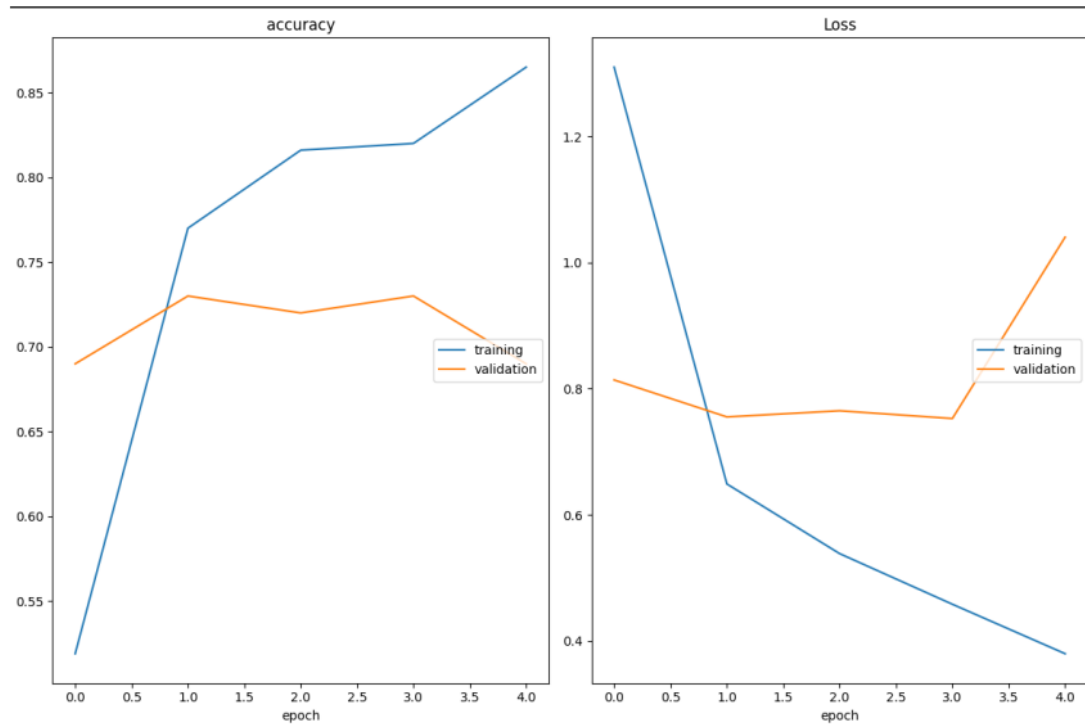


FIGURE 48 RESULTS CONDUCTED FOR EVOLVING POPULATION OVER 5 GENERATIONS

5.2.1 Best Result Case

```
accuracy
  training      (min: 0.519, max: 0.865, cur: 0.865)
  validation    (min: 0.690, max: 0.730, cur: 0.690)
Loss
  training      (min: 0.380, max: 1.310, cur: 0.380)
  validation    (min: 0.753, max: 1.040, cur: 1.040)
Model: "sequential_48"

Layer (type)                 Output Shape              Param #
=====
conv2d_123 (Conv2D)          (None, 28, 28, 49)        784
conv2d_124 (Conv2D)          (None, 28, 28, 39)       15327
max_pooling2d_52 (MaxPooling2D) (None, 14, 14, 39)         0
conv2d_125 (Conv2D)          (None, 14, 14, 66)       41250
flatten_34 (Flatten)         (None, 12936)              0
dense_82 (Dense)             (None, 69)                892653
dense_83 (Dense)             (None, 10)                 700
...
Trainable params: 950714 (3.63 MB)
Non-trainable params: 0 (0.00 Byte)

4/4 [=====] - 0s 5ms/step - loss: 1.0404 - accuracy: 0.6900
```

FIGURE 49 BEST SCORING CANDIDATE

5.2.2 Limitations

This method lacks the skip connections as most of the research happens on the dataset scores best with sequential models

5.3 Evaluation

5.3.1 Accuracy

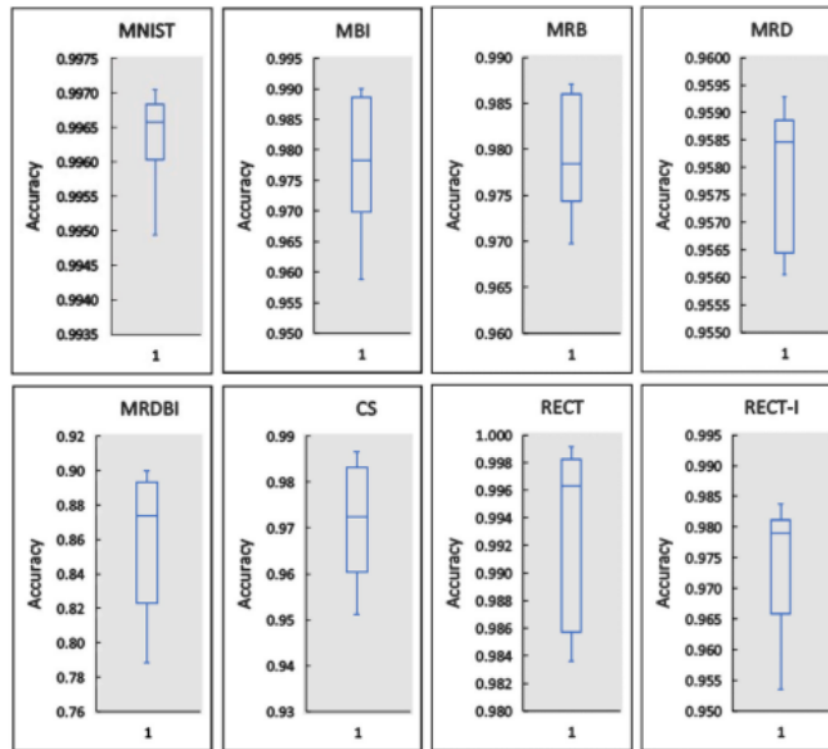


FIGURE 50 ACCURACY BOX PLOTS AFTER EVOLVING CANDIDATES ON EACH DATASET

5.3.2 Time Performance

Time performance could be improved as the time metric could be added to the fitness those evolving faster neural nets overtime.

5.3.3 Comparison with proposed work

Our proposed Solution:

A variable-length encoding strategy was used to represent CNN architectures. It optimized the layers in CNN models, offering a more detailed search. This was accomplished by mutation and crossover operations to evolve CNN through the original DE.

Other Methods:

Model	Search Method	Proposed Work	Limitation	Dataset
GeNet (Xie & Yuille, 2017)	GA	A fixed-length binary encoding strategy that represents CNN, and GA was used to encode connections between layers of the CNN model.	Hyper-parameters of the associated layers were ignored.	CIFAR-10
EvoCNN (Sun et al., 2017b)	GA	A variable-length encoding strategy was used to represent CNN, and GA was used to optimize both connections and	The best-generated architecture faced an over-fitting problem because of considering a	MNIST, Convex, Rectangle

		weights of the CNN model.	large number of parameters.	
MA-NET (Dong et al., 2020)	Memtic	CNN was represented using a variable-length encoding strategy, and the memetic algorithm was used to optimize connections between layers of the CNN model.	Because of the large number of parameters considered, the best-generated architecture may have encountered an over-fitting problem.	MNIST, MNIST variation, Convex, Rectangle
DeepSwarm (Byla & Pang, 2019)	ACO	Pheromone information was used collectively to find the best CNN model. The authors used local and global pheromone update rules during model execution.	Associated collective behavior made the approach computationally expensive.	MNIST, Fashion-MNIST, CIFAR-10
IPPSO (B. Wang et al., 2018b)	PSO	A novel encoding scheme inspired by convolutional	Because of the architecture's fixed pre-	MNIST with noisy image, Convex

		networking to represent the CNN architecture. PSO was used to optimize the layers and associated hyper-parameters of the CNN model.	defined length, the depth of the architectural search space was reduced.	
psoCNN (Fernandes Junior & Yen, 2019)	PSO	A variable-length encoding strategy to represent CNN architecture and layer types, such as Conv, Pool, and FC, was updated by copying the layers in a random fashion from the global or personal best solutions.	The architectural search space that might be less explored because a new particle was built from the global or personal best particle.	MNIST, MNIST variation, Convex, Rectangle
DECNN (B. Wang et al., 2018a)	DE	An internet protocol (IP)-based encoding strategy represented a CNN	Trim operation before the mutation operation may	MNIST, MNIST variation, Convex, Rectangle

		architecture. DE mutation and crossover operations were used to evolve CNN models. An external archive was integrated to generate offspring.	have led to a loss of exploration in the architectural search space, and two crossover operations had to avoid crossover operations and the approach very complicated and expensive.	
DE-NAS (Awad et al., 2020)	DE	A cell-based encoding scheme was used to represent the CNN models. The encoding was mapped to the NAS search space with a	Converting from continuous to discrete space was expensive. Using a cell-based encoding strategy made	CIFAR-10, CIFAR-100, CIFAR-C

		structure-based cell search.	the approach more costly and complicated.	
--	--	------------------------------	---	--

Chapter 6 Conclusion and Future Work

6.1 Conclusion

In this thesis, we developed an Advanced Driver Assistance System (ADAS) using neuro-evolutionary techniques to optimize Convolutional Neural Networks (CNNs) for real-time trajectory prediction and risk assessment. The system integrates data from LIDAR, IMU, and CAN Bus sensors, processes this data through various modules, and provides real-time feedback to the driver via augmented reality (AR) glasses. Our approach demonstrates the potential of combining reinforcement learning with evolutionary algorithms to enhance the performance and adaptability of ADAS. The results indicate that our system can significantly improve driving safety by providing accurate trajectory predictions and timely risk assessments.

In the realm of Advanced Driver Assistance Systems (ADAS), incorporating neuro-evolutionary methods significantly enhances the evaluation of both vehicle trajectories and user thought processes using EEG data. This approach uses evolutionary algorithms to optimize Long Short-Term Memory (LSTM) networks, which are ideal for processing complex time-series data like EEG signals and vehicle trajectories. Traditional methods rely on manually tuning neural network parameters, but neuro-evolutionary methods automate this process, resulting in more efficient and effective models. Evolving LSTM networks ensures improved accuracy in predicting vehicle trajectories and interpreting EEG signals, leading to better-tailored models. This integration allows ADAS to understand driver behavior

and cognitive states, providing adaptive and responsive support for safer and more efficient driving experiences. By combining trajectory prediction with cognitive state analysis, the ADAS can anticipate both the vehicle's movements and the driver's needs, enhancing overall reliability and creating a more holistic driving assistance system.

6.2 Problem Issue

During the development and implementation of the ADAS, several issues were encountered, categorized into technical and scientific issues.

6.2.1 Technical Issues

6.2.1.1 Sensor Integration

Integrating data from multiple sensors (LIDAR, IMU, and CAN Bus) posed synchronization challenges.

Ensuring the real-time processing of sensor data required significant optimization of data pipelines.

6.2.1.2 Model Training

Training the CNN models with large datasets required substantial computational resources.

Overfitting was a persistent issue, necessitating careful tuning of hyperparameters and the use of data augmentation techniques.

6.2.1.3 hardware Constraints

The deployment of the system on embedded hardware required optimizations to ensure efficient resource usage without compromising performance.

Real-time processing demands imposed constraints on the model complexity and required efficient coding practices.

But Neuro Evolution can introduce a faster network as time could be used in fitness evaluation.

6.2.2 Scientific Issue

6.2.2.1 Model Interpretability

Interpreting the decisions made by deep learning models remains a challenge, particularly in safety-critical applications like ADAS.

Understanding the internal workings of the reinforcement learning model and its decision-making process required in-depth analysis.

6.2.2.2 Generalization

Ensuring that the trained models generalize well to different driving environments and conditions was challenging.

The diversity of training data significantly impacted the model's ability to handle edge cases and rare scenarios.

6.2.2.3 Evolutionary Algorithm Limitations

The evolutionary approach, while powerful, introduced stochastic elements that could lead to inconsistent results across different runs.

Balancing exploration and exploitation in the genetic algorithm required careful parameter tuning.

6.3 Future Work

6.3.1 Enhanced Sensor Fusion

Advanced Sensor Fusion Techniques: Explore methods to integrate data from multiple sensors, such as LIDAR, cameras, radar, and IMU, to improve the accuracy and reliability of environmental perception.

Additional Sensors: Investigate the inclusion of additional sensors to enhance the system's perception capabilities, providing a more comprehensive understanding of the environment.

6.3.2 Model Optimization

Computational Overhead Reduction: Develop techniques to reduce the computational demands of CNN models, making them suitable for real-time applications on embedded systems.

Model Compression and Quantization: Explore methods to compress and quantize models, optimizing them for deployment on resource-constrained devices without significant loss of accuracy.

6.3.3 Improved Interpretability

Deep Learning Interpretability: Research methods to improve the interpretability of deep learning models, providing insights into their decision-making processes.

Visualization Tools: Develop visualization tools to better understand the internal workings of reinforcement learning models and their decision-making strategies.

6.3.4 Robustness and Generalization

Diverse Training Datasets: Expand the diversity of training datasets to include a wider range of driving conditions and environments, improving model robustness.

Domain Adaptation Techniques: Investigate techniques to ensure models generalize well to new and unseen driving scenarios.

6.3.5 Integration with Autonomous Systems

ADAS Integration: Explore the integration of the developed ADAS with fully autonomous driving systems, enhancing their decision-making capabilities.

Semi-Autonomous Modes: Research the application of the system in semi-autonomous driving modes, providing advanced driver assistance while maintaining driver control.

6.3.6 User Interface Enhancements

AR Glasses Interface: Improve the user interface of AR glasses to provide more intuitive and user-friendly feedback to the driver.

Haptic and Auditory Feedback: Investigate the use of haptic feedback and auditory signals to complement the visual feedback provided by the AR system.

6.3.7 Introducing Skip Connections with NEAT

Neuro-Evolution of Augmenting Topologies (NEAT): Leverage NEAT to evolve the architecture of neural networks dynamically, optimizing both the topology and weights of the networks.

Skip Connections: Introduce skip connections into the NEAT algorithm to enhance the performance of evolved networks. Skip connections can help mitigate the vanishing gradient problem, facilitate the training of deeper networks, and improve overall model performance.

Implementation Strategy: Modify the NEAT algorithm to allow for the evolution of architectures that include skip connections, enabling the direct transfer of activations from one layer to a subsequent non-adjacent layer.

Benefits: The inclusion of skip connections can lead to more robust networks capable of learning more complex representations, improving the accuracy and generalization of the model.

Experimental Validation: Conduct experiments to validate the effectiveness of skip connections in evolved architectures, comparing performance metrics with and without skip connections to quantify the benefits.

6.3.8 potential linking between CNN and GNN

As NEAT could introduce a=various connection type, (Jiao et al., 2024) have conducted research about linking CNN with GNN those opening the gate for further improvements in Neural Architecture Search

6.3.7 Expanding the Neuro-Evolutionary approach

TCNs and more complex approaches could be used to expand our model capabilities of discovering new architectures efficiently.

References

- Antonante, P., Veer, S., Leung, K., Weng, X., Carlone, L., & Pavone, M. (2023). *Task-Aware Risk Estimation of Perception Failures for Autonomous Vehicles*. <http://arxiv.org/abs/2305.01870>
- Awad, N., Mallik, N., & Hutter, F. (2020). *Differential Evolution for Neural Architecture Search*. <http://arxiv.org/abs/2012.06400>
- Baig, M. Z., Aslam, N., Shum, H. P. H., & Zhang, L. (2017). Differential evolution algorithm as a tool for optimal feature subset selection in motor imagery EEG. *Expert Systems with Applications*, 90, 184–195. <https://doi.org/https://doi.org/10.1016/j.eswa.2017.07.033>
- Bhariliya, V., & Kumar, N. (2023a). *Machine Learning for Autonomous Vehicle's Trajectory Prediction: A comprehensive survey, Challenges, and Future Research Directions*. <http://arxiv.org/abs/2307.07527>
- Bhariliya, V., & Kumar, N. (2023b). *Machine Learning for Autonomous Vehicle's Trajectory Prediction: A comprehensive survey, Challenges, and Future Research Directions*. <http://arxiv.org/abs/2307.07527>
- Bohrer, J. da S., Grisci, B. I., & Dorn, M. (2020). *Neuroevolution of Neural Network Architectures Using CoDeepNEAT and Keras*. <http://arxiv.org/abs/2002.04634>
- Byla, E., & Pang, W. (2019). *DeepSwarm: Optimising Convolutional Neural Networks using Swarm Intelligence*. <http://arxiv.org/abs/1905.07350>
- Chen, W., Subramanian, D., & Paternain, S. (2023). *Probabilistic Constraint for Safety-Critical Reinforcement Learning*. <http://arxiv.org/abs/2306.17279>
- Chen, Y., Karkus, P., Ivanovic, B., Weng, X., & Pavone, M. (2023). *Tree-structured Policy Planning with Learned Behavior Models*. <http://arxiv.org/abs/2301.11902>
- Cybenkot, G. (1989). Mathematics of Control, Signals, and Systems Approximation by Superpositions of a Sigmoidal Function*. In *Math. Control Signals Systems* (Vol. 2).
- Das, S., & Suganthan, P. N. (2011). Differential evolution: A survey of the state-of-the-art. *IEEE Transactions on Evolutionary Computation*, 15(1), 4–31. <https://doi.org/10.1109/TEVC.2010.2059031>
- Dong, J., Zhang, L., Hou, B., & Feng, L. (2020). A Memetic Algorithm for Evolving Deep Convolutional Neural Network in Image Classification. *2020 IEEE Symposium Series on Computational Intelligence, SSCI 2020*, 2663–2669. <https://doi.org/10.1109/SSCI47803.2020.9308162>
- Duan, Y., Wang, Z., Li, Y., Tang, J., Wang, Y.-K., & Lin, C.-T. (2022). *Cross Task Neural Architecture Search for EEG Signal Classifications*. <http://arxiv.org/abs/2210.06298>
- Fernandes Junior, F. E., & Yen, G. G. (2019). Particle swarm optimization of deep neural networks architectures for image classification. *Swarm and Evolutionary Computation*, 49, 62–74. <https://doi.org/10.1016/j.swevo.2019.05.010>

- Hanin, B., & Sellke, M. (2017). *Approximating Continuous Functions by ReLU Nets of Minimal Width*. <http://arxiv.org/abs/1710.11278>
- Hornik, K. (1991). Approximation Capabilities of Multilayer Feedforward Networks. In *Neural Networks* (Vol. 4).
- Humeniuk, D., Khomh, F., & Antoniol, G. (2023). Reinforcement learning informed evolutionary search for autonomous systems testing. *ArXiv Preprint ArXiv:2308.12762*.
- Ingolfsson, T. M., Hersche, M., Wang, X., Kobayashi, N., Cavigelli, L., & Benini, L. (2020). *EEG-TCNet: An Accurate Temporal Convolutional Network for Embedded Motor-Imagery Brain-Machine Interfaces*. <http://arxiv.org/abs/2006.00622>
- Jiang, J., Zhang, Z., Xu, C., Yu, Z., & Peng, Y. (2023). *One Forward is Enough for Neural Network Training via Likelihood Ratio Method*. <http://arxiv.org/abs/2305.08960>
- Jiao, Z., Zhang, H., & Li, X. (2024). *CNN2GNN: How to Bridge CNN with GNN*. <http://arxiv.org/abs/2404.14822>
- Leshno, M., Lin, V. Y., & Pinkus, A. (1992). *MULTILAYER FEEDFORWARD NETWORKS WITH A NON-POLYNOMIAL ACTIVATION FUNCTION CAN APPROXIMATE ANY FUNCTION*.
- Li, Q., Ou, B., Liang, Y., Wang, Y., Yang, X., & Li, L. (2023). TCN-SA: A Social Attention Network Based on Temporal Convolutional Network for Vehicle Trajectory Prediction. *Journal of Advanced Transportation, 2023*. <https://doi.org/10.1155/2023/1286977>
- Liao, H., Li, Z., Shen, H., Zeng, W., Liao, D., Li, G., Li, S. E., & Xu, C. (2023). *BAT: Behavior-Aware Human-Like Trajectory Prediction for Autonomous Driving*. <http://arxiv.org/abs/2312.06371>
- Lin, B., Bouneffouf, D., Cecchi, G., Baihan Lin, st, & Djallel Bouneffouf, nd. (2022). *Predicting Human Decision Making with LSTM*. <https://hal.science/hal-03683845>
- Liu, H., Simonyan, K., & Yang, Y. (2018). *DARTS: Differentiable Architecture Search*. <http://arxiv.org/abs/1806.09055>
- Maul, P., Mueller, M., Enkler, F., Pigova, E., Fischer, T., & Stamatogiannakis, L. (n.d.). *BeamNG.tech Technical Paper*. <https://github.com/BeamNG/BeamNGpy>
- Padfield, N., Zabalza, J., Zhao, H., Masero, V., & Ren, J. (2019). EEG-based brain-computer interfaces using motor-imagery: Techniques and challenges. *Sensors (Switzerland), 19*(6). <https://doi.org/10.3390/s19061423>
- Stanley, K. O., Clune, J., Lehman, J., & Miikkulainen, R. (2019). Designing neural networks through neuroevolution. In *Nature Machine Intelligence* (Vol. 1, Issue 1, pp. 24–35). Nature Research. <https://doi.org/10.1038/s42256-018-0006-z>
- Stanley, K. O., & Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation, 10*(2), 99–127. <https://doi.org/10.1162/106365602320169811>
- Stapleton, F., Galván, E., Sistu, G., & Yogamani, S. (2022). *Neuroevolutionary Multi-objective approaches to Trajectory Prediction in Autonomous Vehicles*. <http://arxiv.org/abs/2205.02105>

- Sun, Y., Xue, B., Zhang, M., & Yen, G. G. (2017a). *Evolving Deep Convolutional Neural Networks for Image Classification*. <http://arxiv.org/abs/1710.10741>
- Sun, Y., Xue, B., Zhang, M., & Yen, G. G. (2017b). *Evolving Deep Convolutional Neural Networks for Image Classification*. <http://arxiv.org/abs/1710.10741>
- Talbi, E.-G. (2009). *Metaheuristics : from design to implementation*. John Wiley & Sons.
- Ünal, H. T., & Başçiftçi, F. (2022). Evolutionary design of neural network architectures: a review of three decades of research. *Artificial Intelligence Review*, 55(3), 1723–1802.
<https://doi.org/10.1007/s10462-021-10049-5>
- Wang, B., Sun, Y., Xue, B., & Zhang, M. (2018a). *A Hybrid Differential Evolution Approach to Designing Deep Convolutional Neural Networks for Image Classification*. <http://arxiv.org/abs/1808.06661>
- Wang, B., Sun, Y., Xue, B., & Zhang, M. (2018b). *Evolving Deep Convolutional Neural Networks by Variable-length Particle Swarm Optimization for Image Classification*.
<http://arxiv.org/abs/1803.06492>
- Wang, L., Zhao, M., Liu, E., Sun, K., & Cheng, R. (2024). *Tensorized NeuroEvolution of Augmenting Topologies for GPU Acceleration*. <http://arxiv.org/abs/2404.01817>
- Xie, L., & Yuille, A. (2017). *Genetic CNN*. <http://arxiv.org/abs/1703.01513>
- Zeng, W., Luo, W., Suo, S., Sadat, A., Yang, B., Casas, S., & Urtasun, R. (2021). *End-to-end Interpretable Neural Motion Planner*. <http://arxiv.org/abs/2101.06679>